

Equazione del Calore 2D

e Fattorizzazioni di Cholesky

Libro di teoria e implementazione

Informatica con Laboratorio — A.A. 2025/2026

Giosuè Aiello

Sommario

Questo documento è una guida completa alla comprensione del progetto, dalla matematica alla implementazione. Per ogni concetto teorico si fornisce la derivazione passo per passo, il collegamento al codice sorgente C++ e Python, e intuizioni fisiche che rendono la materia concreta. Lo scopo è che tu possa rileggere questo testo e comprendere *perché* ogni riga di codice è scritta in quel modo.

Indice

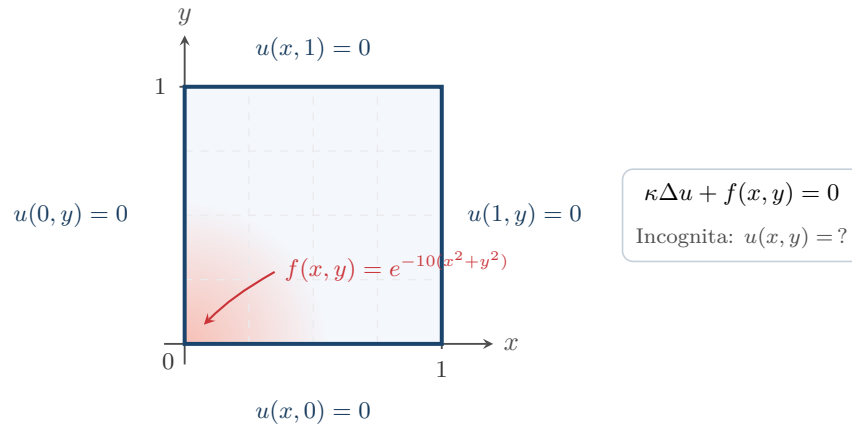
1	Il problema fisico: l'equazione del calore	3
1.1	Cosa stiamo risolvendo	3
1.2	Condizioni al contorno di Dirichlet	3
1.3	Perché non si risolve analiticamente	4
2	Discretizzazione: dalle equazioni alle matrici	5
2.1	La griglia di calcolo	5
2.2	Approssimazione delle derivate: lo stencil a 5 punti	6
2.3	Dalla stencil al sistema lineare $Au = b$	7
2.4	Proprietà della matrice A — perché importano	8
3	La fattorizzazione di Cholesky	10
3.1	Cos'è e quando si usa	10
3.2	Il fenomeno del fill-in	10
3.3	La Nested Dissection: Teoria, Algoritmo e Vantaggi	11
3.3.1	Principio di Funzionamento (La scomposizione del Grafo)	11
3.3.2	Perché la Nested Dissection minimizza il fill-in?	12
3.3.3	Due scuole di pensiero: Approccio su Grafo vs Approccio Geometrico	13
3.3.4	Come si applica nel Progetto (C++ e Python)	13
3.4	Esempio grafico con un sistema minimo a 3 nodi	14
4	Modulo 1 — <code>task1_grid.cpp</code>: griglia e grafo	15
4.1	Obiettivo e panoramica del flusso	15
4.2	Lettura dell'argomento da riga di comando: <code>argc</code> , <code>argv</code> e <code>stoi</code>	15
4.3	Creazione della directory di output: <code>mkdir</code> e il codice 0755	16
4.4	La funzione <code>getNodeId</code> : <code>inline</code> e formula row-major	16
4.5	Generazione coordinate e precisione floating point	17
4.6	Costruzione del grafo con <code>reserve</code> e approccio “in avanti”	17
4.7	Scrittura degli archi senza duplicati: la condizione $u < v$	18

4.8	Gestione degli Errori (Fail) e Output Generati	18
5	Modulo 2 — <code>task2_reorder.cpp</code>: nested dissection	19
5.1	Panoramica: approccio geometrico vs approccio su grafo	19
5.2	La struttura <code>Node</code> : perché non usare una coppia di interi?	19
5.3	La struttura <code>StackItem</code> e il flag <code>is_separator</code>	19
5.4	Versione iterativa con <code>std::stack</code> : perché non la ricorsione?	19
5.5	Il partizionamento geometrico: mediana intera e <code>reserve</code> pessimistico	21
5.6	L'alternanza degli assi e la logica LIFO dello stack	21
5.7	Gestione degli Errori (Fail) e Output Generato	22
6	Modulo 3 — <code>task3_assemble.cpp</code>: sistema lineare	23
6.1	Panoramica: cosa fa il Modulo 3 e perché è il più complesso	23
6.2	Costanti fisiche: <code>static const</code> e pre-calcolo dei coefficienti	23
6.3	Le due mappe di permutazione: il contratto tra Modulo 2 e Modulo 3	23
6.4	Gli array di spostamento <code>di[]</code> e <code>dj[]</code> : il ciclo compatto sui 4 vicini	24
6.5	Il ciclo di assemblaggio completo con gestione del bordo	24
6.6	La struttura <code>Triplet</code> e il formato COO di <code>A.txt</code>	25
6.7	Le condizioni al bordo: <code>switch</code> e funzioni di verifica	26
6.8	Gestione degli Errori (Fail) e Output Generati	26
7	Modulo 4 — <code>solver.ipynb</code>: risoluzione Python	28
7.1	Dal formato COO a CSC: perché due formati diversi?	28
7.2	Lettura del vettore <code>b.txt</code> e delle coordinate	28
7.3	La fattorizzazione di Cholesky con CHOLMOD: API e scelte	29
7.4	La risoluzione del sistema: <code>spsolve_triangular</code> vs <code>spsolve</code>	29
7.5	Automazione della pipeline: <code>subprocess</code> e iniezione via <code>stdin</code>	30
7.6	Interpretazione dei grafici e analisi empirica	31
8	Riepilogo: dalla teoria al codice	32
9	Appendice: Domande Ricorrenti d'Esame	33
9.1	1. Struttura generale del codice: come sono suddivisi i programmi e cosa fanno i diversi moduli?	33
9.2	2. Ordinamento: approccio a grafi vs geometrico e dettagli dell'algoritmo	33
9.3	3. Il generatore di matrici sparse: assemblaggio e struttura printata	34
9.4	4. Soluzione grafica (Color Plot/Heatmap)	34
9.5	5. Mostrare il codice in azione	34
9.6	6. Generazione via Intelligenza Artificiale (Meta-Commento Documentale)	34
9.7	7. Dettagli Implementativi C++ e Scelte Sintattiche	35

1 Il problema fisico: l'equazione del calore

1.1 Cosa stiamo risolvendo

Immagina una **piastra quadrata di metallo** di dimensioni 1×1 m. I quattro bordi sono tenuti a temperatura fissa (ad esempio zero, come se fossero a contatto con il ghiaccio). All'interno, vicino all'angolo in basso a sinistra, è acceso un piccolo elemento riscaldante che emette calore seguendo una distribuzione gaussiana.



Dominio $\Omega = [0, 1]^2$ con la griglia di discretizzazione $h = 0.25$ (tratteggio). La sorgente di calore gaussiana $f(x, y)$ (in rosso) riscalda l'angolo $(0, 0)$; sui bordi si impongono condizioni di Dirichlet omogenee $u = 0$.

Vogliamo trovare **l'unica funzione** $u(x, y)$ che, a regime stazionario (nessuna variazione nel tempo), soddisfa l'equazione di Poisson:

$$\kappa \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) + f(x, y) = 0, \quad (x, y) \in \Omega = [0, 1]^2. \quad (1)$$

Concetto chiave

Significato dei simboli:

- $u(x, y)$: **temperatura** nel punto (x, y) — la nostra incognita.
- $\kappa = 0.01$: **diffusività termica** del materiale. Più è alta, più il calore si propaga; con κ piccolo il calore resta concentrato vicino alla sorgente.
- $\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}$: il **Laplaciano**. Misura quanto u in un punto *differisce dalla media dei suoi vicini infinitesimi*. Se $\nabla^2 u > 0$, quel punto è più freddo dei vicini (e riceve calore per conduzione); se $\nabla^2 u < 0$, è più caldo.
- $f(x, y) = e^{-10(x^2 + y^2)}$: **sorgente di calore**. Una gaussiana centrata nell'origine: $f(0, 0) = 1$ (massimo), $f(1, 1) \approx 2 \times 10^{-9}$ (praticamente zero).

Interpretazione dell'equazione: all'equilibrio, $\kappa \nabla^2 u = -f$, cioè la diffusione bilancia esattamente la sorgente puntuale. Dove $f > 0$ la sorgente *inietta* calore; l'equazione obbliga il Laplaciano ad essere negativo in quei punti (zona più calda dei vicini).

1.2 Condizioni al contorno di Dirichlet

L'equazione (1) da sola ha *infinite* soluzioni (si può aggiungere qualsiasi soluzione dell'equazione omogenea). Le **condizioni al contorno di Dirichlet** fissano la temperatura sul bordo $\partial\Omega$ e rendono la soluzione **unica**:

$$u(x, y) = u_0(x, y), \quad (x, y) \in \partial\Omega.$$

Nel caso base si assume $u_0 \equiv 0$ (*bordo freddo*). Il progetto implementa anche cinque altre scelte di u_0 , utili per verificare la correttezza del codice (cfr. Sezione 6).

Osservazione 1.1. Se $f = 0$ e $u_0 = 0$, l'unica soluzione è $u \equiv 0$. Questo è il classico *sanity check*: se il codice dà un risultato non nullo in questo caso, c'è un bug.

1.3 Perché non si risolve analiticamente

Dal punto di vista teorico, l'equazione di Poisson $\kappa\Delta u + f(x, y) = 0$ su un dominio rettangolare con condizioni di Dirichlet omogenee ammette una soluzione analitica in forma di serie. Utilizzando il metodo di separazione delle variabili, la temperatura $u(x, y)$ può essere espressa esplicitamente come una doppia serie di Fourier di seni:

$$u(x, y) = \sum_{m=1}^{\infty} \sum_{n=1}^{\infty} C_{m,n} \sin(m\pi x) \sin(n\pi y) \quad (2)$$

dove i coefficienti $C_{m,n}$ si ricavano proiettando la sorgente termica $f(x, y)$ sulla base trigonometrica:

$$C_{m,n} = \frac{4}{\kappa\pi^2(m^2 + n^2)} \int_0^1 \int_0^1 f(x, y) \sin(m\pi x) \sin(n\pi y) dx dy \quad (3)$$

Tuttavia, nel nostro caso specifico si presentano due ostacoli matematici e computazionali che impediscono di ottenere una **forma chiusa** utilizzabile nella pratica:

- **Assenza di primitive elementari:** La sorgente termica assegnata è una funzione gaussiana $f(x, y) = e^{-10(x^2+y^2)}$. L'integrale definito di una gaussiana moltiplicata per funzioni trigonometriche su un dominio limitato $[0, 1]^2$ non possiede una primitiva esprimibile in termini di funzioni elementari. La sua valutazione formale rimanda all'uso di funzioni speciali (come la funzione degli errori erf) che non si traducono in una formula algebrica compatta.
- **Lenta convergenza della serie:** Anche ipotizzando di calcolare numericamente i coefficienti $C_{m,n}$, la soluzione rimarrebbe una sommatoria di infiniti termini. Poiché la nostra sorgente gaussiana è fortemente localizzata in prossimità dell'origine (decresce molto rapidamente a causa del fattore -10), la serie trigonometrica convergerebbe in modo estremamente lento. Sarebbero necessarie migliaia di armoniche ad alta frequenza per ricostruire accuratamente il picco di calore senza introdurre oscillazioni spurie.

Dall'analisi matematica all'algebra lineare numerica Data l'intrattabilità e l'inefficienza dell'approccio analitico continuo, si deve cambiare completamente paradigma ricorrendo a un'approssimazione numerica. Tramite la **discretizzazione alle differenze finite**, si restringe la valutazione della soluzione a una griglia discreta di punti sul dominio.

Le derivate parziali del Laplaciano vengono rimpiazzate da semplici rapporti incrementali tra i nodi vicini. Questa operazione cruciale trasforma un problema integro-differenziale a dimensione infinita in un **sistema lineare di equazioni algebriche**:

$$A\mathbf{u} = \mathbf{b} \quad (4)$$

dove l'incognita non è più una funzione analitica continua, ma un vettore $\mathbf{u} \in \mathbb{R}^{N^2}$ che contiene le temperature approssimate nei nodi della griglia. Questo approccio permette infine di sfruttare la potenza dei calcolatori e le tecniche avanzate dell'algebra lineare (come le fattorizzazioni di Cholesky per matrici sparse discusse in questo elaborato) per giungere alla soluzione in tempi rapidi.

2 Discretizzazione: dalle equazioni alle matrici

2.1 La griglia di calcolo

Per risolvere l'equazione stazionaria del calore sul calcolatore, è necessario convertire il problema differenziale continuo in un problema algebrico discreto. Il primo passo di questo processo, noto come *discretizzazione spaziale*, consiste nel sostituire l'infinità di punti del dominio continuo $\Omega = [0, 1]^2$ con un insieme finito di punti di campionamento.

Suddividiamo quindi il dominio in una griglia cartesiana uniforme di $(N + 2) \times (N + 2)$ punti totali, caratterizzata da un passo spaziale costante h :

$$h = \frac{1}{N + 1}.$$

I singoli punti della griglia (o nodi) sono identificati dalle coordinate spaziali discrete $(x_i, y_j) = (ih, jh)$, dove gli indici i e j variano in $\{0, 1, \dots, N + 1\}$.

Dal punto di vista fisico e matematico, i nodi della griglia si dividono in due categorie distinte:

- **Punti di bordo** ($i = 0, i = N + 1, j = 0$, oppure $j = N + 1$): Rappresentano la frontiera fisica del dominio ($\partial\Omega$). In questi nodi la temperatura è già nota a priori grazie alle condizioni al contorno di Dirichlet ($u(x_i, y_j) = u_0$). Poiché questi valori sono noti, non costituiscono delle incognite per il sistema, ma andranno a modificare il termine noto b .
- **Punti interni** ($1 \leq i, j \leq N$): Rappresentano l'interno della piastra metallica, dove la temperatura $u_{i,j} \approx u(x_i, y_j)$ è incognita e deve essere determinata. Essendo presenti N nodi interni lungo l'asse x e N lungo l'asse y , il problema presenta in totale N^2 *incognite*.

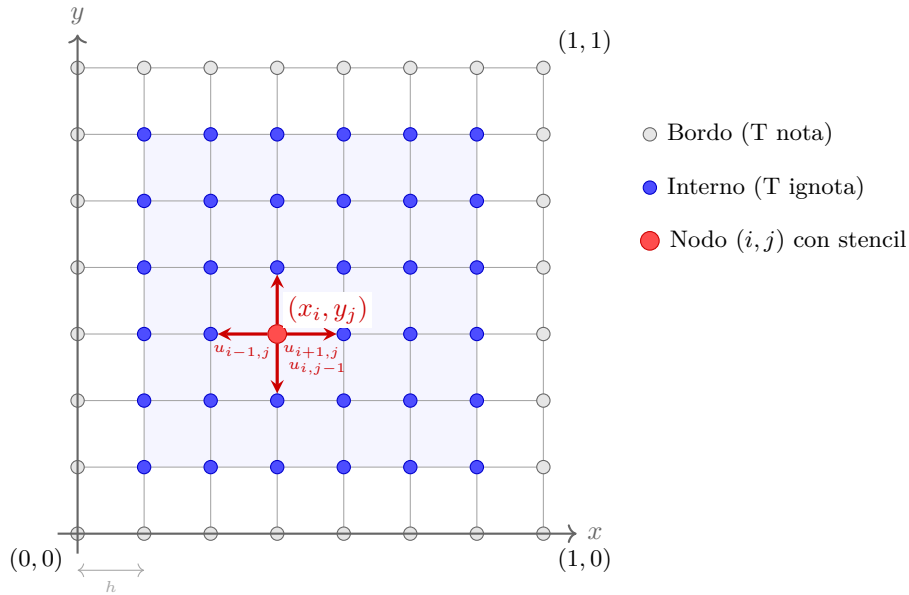


Figura 1: Griglia di discretizzazione per $N = 6$ ($h = 1/7$). In rosso il nodo (x_i, y_j) con le frecce verso i quattro vicini dello stencil. I nodi di bordo (grigio) hanno temperatura nota; i nodi interni (blu) sono le N^2 incognite.

Poiché i risolutori di sistemi lineari lavorano con vettori unidimensionali di incognite, dobbiamo definire un criterio per ordinare i nodi interni bidimensionali in una sequenza monodimensionale. A questo scopo introduciamo una mappa di indicizzazione biunivoca ϕ , che associa a ciascuna coordinata di griglia (i, j) un identificatore intero univoco (ID) compreso tra 0 e $N^2 - 1$:

$$\phi(i, j) = (i - 1) \cdot N + (j - 1), \quad \phi : \{1, \dots, N\}^2 \rightarrow \{0, \dots, N^2 - 1\}.$$

Questa mappa adotta un *ordinamento naturale per righe* (o ordine lessicografico): i nodi interni vengono numerati da sinistra a destra, riga dopo riga, procedendo dal basso verso l'alto del dominio.

Esempio 2.1. Ipotezziamo una griglia con $N = 3$, che genera $3 \times 3 = 9$ incognite interne. Per identificare la posizione nel vettore della temperatura corrispondente al nodo posizionato alla seconda riga e terza colonna della griglia interna, ovvero $(i = 2, j = 3)$, applichiamo la mappa ϕ :

$$\phi(2, 3) = (2 - 1) \cdot 3 + (3 - 1) = 1 \cdot 3 + 2 = 5.$$

Questo nodo corrisponde al sesto elemento del vettore delle incognite (essendo l'indicizzazione basata su 0). Nel codice di progetto, questo calcolo è centralizzato e implementato in modo efficiente dalla funzione `getNodeId(i, j, N)`.

2.2 Approssimazione delle derivate: lo stencil a 5 punti

L'idea cardine del metodo delle differenze finite consiste nel sostituire le derivate continue presenti nell'operatore differenziale con opportune combinazioni algebriche discrete dei valori della funzione nei nodi della griglia. In questo modo si trasforma un'equazione differenziale (che richiede la conoscenza della funzione in infiniti punti) in un'equazione algebrica locale.

Per comprendere l'origine di questa approssimazione, consideriamo inizialmente una generica funzione monodimensionale sufficientemente regolare $g(x)$. La sua derivata seconda in un punto generico x_i può essere approssimata utilizzando lo schema classico delle **differenze finite centrate del secondo ordine**:

$$g''(x_i) \approx \frac{g(x_{i-1}) - 2g(x_i) + g(x_{i+1}))}{h^2}.$$

Concetto chiave

Derivazione formale via sviluppi di Taylor — perché funziona.

Per dimostrare la validità e valutare l'accuratezza di questa approssimazione, espandiamo la funzione g in serie di Taylor nell'intorno del punto x_i , valutandola nei due nodi adiacenti $x_i + h$ e $x_i - h$:

$$g(x_i + h) = g(x_i) + h g'(x_i) + \frac{h^2}{2} g''(x_i) + \frac{h^3}{6} g'''(x_i) + \mathcal{O}(h^4)$$

$$g(x_i - h) = g(x_i) - h g'(x_i) + \frac{h^2}{2} g''(x_i) - \frac{h^3}{6} g'''(x_i) + \mathcal{O}(h^4)$$

Sommando membro a membro queste due relazioni, osserviamo che i termini associati alle derivate di ordine dispari (g' e g''') si cancellano reciprocamente a causa della simmetria geometrica dei punti rispetto a x_i :

$$g(x_i + h) + g(x_i - h) = 2g(x_i) + h^2 g''(x_i) + \mathcal{O}(h^4).$$

Isolando ora il termine di derivata seconda $g''(x_i)$, otteniamo:

$$g''(x_i) = \frac{g(x_i - h) - 2g(x_i) + g(x_i + h))}{h^2} + \mathcal{O}(h^2).$$

L'**errore di troncamento** locale commesso è dominato dal termine $\mathcal{O}(h^2)$. Questo significa che lo schema è *del secondo ordine*: dimezzando il passo di griglia h (ovvero raddoppiando all'incirca la risoluzione N), l'errore di approssimazione della derivata si riduce di ben quattro volte.

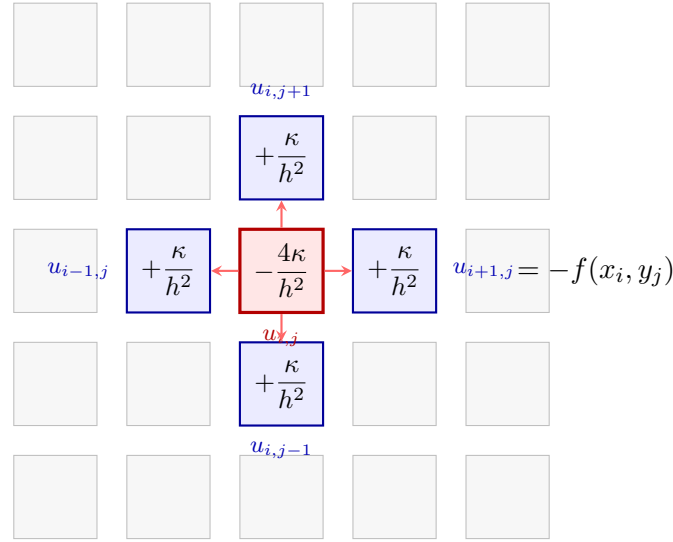
Estendiamo ora questo approccio al caso bidimensionale per approssimare l'operatore di Laplace $\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}$. Applicando lo schema unidimensionale appena derivato in modo indipendente lungo l'asse x e lungo l'asse y , otteniamo:

$$\nabla^2 u|_{(x_i, y_j)} = \frac{\partial^2 u}{\partial x^2}|_{(i, j)} + \frac{\partial^2 u}{\partial y^2}|_{(i, j)} \approx \frac{u_{i-1, j} + u_{i+1, j} + u_{i, j-1} + u_{i, j+1} - 4u_{i, j}}{h^2}.$$

Sostituendo questa approssimazione discretizzata direttamente nell'equazione stazionaria del calore originaria ($\kappa \nabla^2 u + f = 0$), otteniamo l'equazione algebrica locale per il nodo (i, j) :

$$\frac{\kappa}{h^2} (u_{i-1, j} + u_{i+1, j} + u_{i, j-1} + u_{i, j+1} - 4u_{i, j}) + f(x_i, y_j) = 0. \quad (5)$$

Questa formula descrive il celebre **stencil a 5 punti**: l'equazione di bilancio termico in ogni nodo interno (i, j) non richiede la conoscenza globale del dominio, ma coinvolge esclusivamente la temperatura del nodo stesso e quella dei suoi quattro vicini cardinali (Ovest, Est, Sud, Nord).



Maschera dello stencil a 5 punti. Il coefficiente del nodo centrale (rosso) è $-4\kappa/h^2$, a testimoniare che esso dissipa calore verso l'esterno; i quattro vicini (blu) contribuiscono ciascuno con un coefficiente $+\kappa/h^2$. La combinazione lineare deve uguagliare il termine sorgente cambiato di segno $-f(x_i, y_j)$.

2.3 Dalla stencil al sistema lineare $Au = b$

Scrivendo l'equazione locale (5) per ciascuno degli N^2 nodi interni della griglia, si genera un sistema di equazioni algebriche lineari interconnesse. Questo sistema può essere espresso in forma matriciale compatta come:

$$A \mathbf{u} = \mathbf{b}, \quad A \in \mathbb{R}^{N^2 \times N^2}, \quad \mathbf{u}, \mathbf{b} \in \mathbb{R}^{N^2}.$$

In questa formulazione, $\mathbf{u}$ rappresenta il vettore colonna delle temperature incognite ordinate secondo la mappa ϕ , mentre la struttura della matrice dei coefficienti A e del vettore del termine noto $\mathbf{b}$ è governata, riga per riga (con indice di riga $k = \phi(i, j)$), dalle seguenti regole:

Tipo di entrata	Valore	Condizione di applicabilità
$A_{k,k}$ (diagonale principale)	$-\frac{4\kappa}{h^2}$	Sempre (coefficiente del nodo centrale $u_{i,j}$)
$A_{k,k'}$ (fuori diagonale)	$+\frac{\kappa}{h^2}$	Se il vicino $k' = \phi(i', j')$ è un <i>nodo interno</i>
b_k (termine noto, base)	$-f(x_i, y_j)$	Contributo standard della sorgente di calore
$b_k^{\text{bordo}} = -\frac{\kappa}{h^2} u_0$	Correzione di bordo	Per ogni vicino che giace sul <i>bordo</i> del dominio

Trattamento rigoroso delle condizioni al contorno di Dirichlet.

Un punto critico riguarda l'interazione dello stencil con la frontiera fisica della griglia. Quando scriviamo l'equazione per un nodo adiacente al bordo (ad esempio il nodo $(1, j)$, il cui vicino sinistro sarebbe $(0, j)$), il valore di temperatura associato a tale vicino non è un'incognita, bensì un valore fisso e noto pari a u_0 .

** Nota: il simbolo $- =$ va letto come "aggiungo al termine noto un contributo con segno negativo"; equivale a dire che il valore noto u_0 produce una correzione sottrattiva sul secondo membro."*

Per preservare la quadratura della matrice A (che deve rimanere rigorosamente di dimensione $N^2 \times N^2$, contenendo solo le incognite reali), il termine noto associato al bordo viene isolato e spostato al secondo membro dell'equazione. Matematicamente, l'addendo noto $\frac{\kappa}{h^2} u_0$ passa a destra dell'uguale cambiando di segno, traducendosi in una correzione sottrattiva sul termine noto della riga k -esima:

$$b_k^{\text{bordo}} = -\frac{\kappa}{h^2} \cdot u_0(x_{\text{bordo}}, y_{\text{bordo}}).$$

Grazie a questa correzione, il vettore $\mathbf{b}$ assorbe al suo interno tutte le informazioni fisiche relative alle condizioni termiche imposte sulla frontiera del dominio.

2.4 Proprietà della matrice A — perché importano

Concetto chiave

La matrice dei coefficienti $A \in \mathbb{R}^{N^2 \times N^2}$ che otteniamo dalla discretizzazione non è una matrice qualunque, ma possiede quattro proprietà matematiche fondamentali per la risoluzione del sistema:

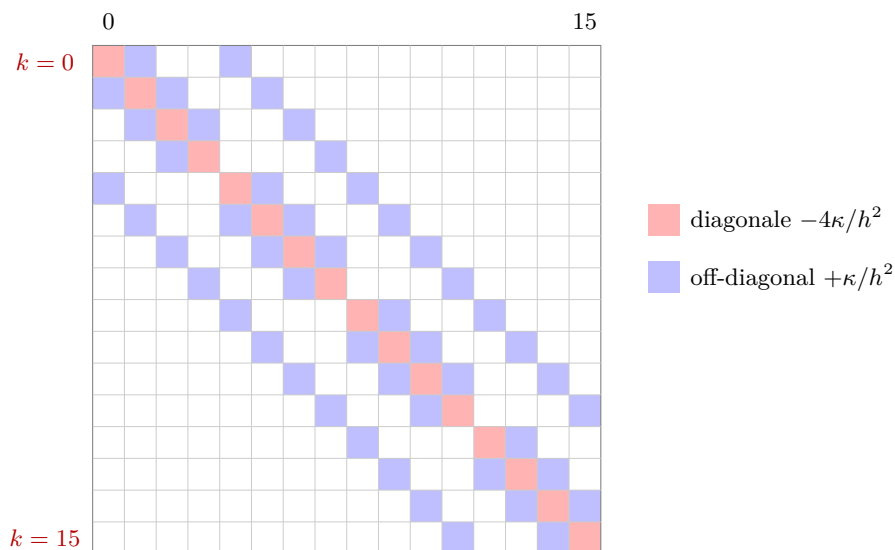
1. **Sparsità estrema:** Quasi tutta la matrice è vuota (piena di zeri). Poiché l'equazione per un singolo nodo coinvolge solo se stesso e i suoi 4 vicini, ogni riga ha al massimo 5 valori non nulli. Di conseguenza, su un totale enorme di N^4 elementi, solo circa $5N^2$ (quindi $\mathcal{O}(N^2)$) sono effettivi. Questo ci obbligherà a usare strutture dati "sparse" per non esaurire la memoria.
2. **Simmetria** ($A = A^T$): Fisicamente, lo scambio di calore è reciproco. L'influenza termica che il nodo k esercita sul nodo k' è identica all'influenza che k' esercita su k (entrambe regolate dal coefficiente κ/h^2). Matematicamente, questo fa sì che la matrice sia perfettamente speculare rispetto alla sua diagonale principale.
3. **Definitezza negativa:** Tutti gli autovalori della matrice A sono strettamente negativi ($\lambda_i < 0$). Il risvolto pratico è che, semplicemente cambiando segno all'intero sistema, la matrice $-A$ risulta **Simmetrica Definita Positiva (SPD)**. Si tratta di una proprietà "d'oro" in algebra lineare, perché ci garantisce di poter usare algoritmi molto veloci e stabili (come la fattorizzazione di Cholesky).
4. **Dominanza diagonale:** In ogni riga, il valore centrale (sulla diagonale) è in valore assoluto forte almeno quanto la somma di tutti gli altri coefficienti della riga ($|-4\kappa/h^2| \geq \sum |\kappa/h^2|$).

- Per i nodi puramente interni si ha un *pareggio perfetto* (il centro vale 4, e i 4 vicini valgono 1 ciascuno).
- Per i nodi adiacenti ai bordi fisici del dominio, invece, la dominanza diventa *stretta* (il segno è $>$). Questo accade perché i collegamenti "verso l'esterno" vengono tagliati via e inglobati nel termine noto, lasciando la riga con meno vicini contro cui competere (es. il centro vale 4, ma ci sono solo 3 vicini, quindi $4 > 3$).

Queste proprietà non sono semplici curiosità teoriche, ma guidano la progettazione dell'algoritmo risolutivo:

- La **sparsità** ci impone di evitare i metodi di memorizzazione densi (che saturerebbero rapidamente la memoria RAM anche per griglie modeste) in favore di strutture dati sparse (come il formato CSR o tripletta), che memorizzano unicamente gli elementi non nulli.
- Il fatto che $-A$ sia una matrice **SPD** è il prerequisito matematico indispensabile che garantisce l'applicabilità, l'esistenza, l'unicità e la stabilità numerica della **fattorizzazione di Cholesky** (nella forma $-A = LL^T$), evitando la necessità di effettuare tecniche di pivoting durante il calcolo.

Organizzazione strutturale di A per $N = 4$ (16×16 incognite, banda = $N = 4$):



Visualizzazione grafica (*spy plot*) della struttura di sparsità di A per $N = 4$. È evidente l'organizzazione a bande della matrice, con la diagonale principale (rosso) affiancata da due sottobande a distanza 1 (accoppiamento orizzontale lungo y) e due bande esterne a distanza $N = 4$ (accoppiamento verticale lungo x). Le periodiche interruzioni lungo le sottobande diagonali rappresentano fisicamente i cambi di riga nella griglia di calcolo, punti in cui non vi è continuità fisica tra l'ultimo nodo di una riga e il primo della successiva.

3 La fattorizzazione di Cholesky

3.1 Cos'è e quando si usa

Poiché $-A$ è **simmetrica definita positiva** (SPD), possiamo calcolare la sua *fattorizzazione di Cholesky*:

$$-A = L L^T,$$

dove L è una matrice **triangolare inferiore** con elementi diagonali positivi, e L^T è la sua trasposta (triangolare superiore).

Concetto chiave

Perché Cholesky invece di Gauss?

Per una matrice SPD, Cholesky è **più efficiente** (circa la metà delle operazioni) e numericamente **più stabile** (non richiede pivoting). Sfrutta la simmetria: invece di calcolare L e U separatamente come LU, si calcola solo L perché $U = L^T$.

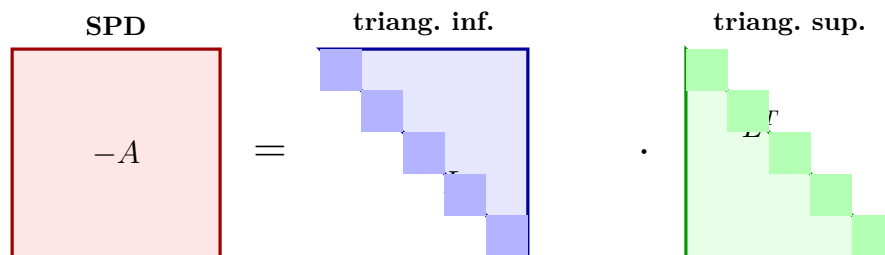
Come si usa per risolvere il sistema?

Risolvere $Au = b$ equivale a $-Au = -b$, cioè $LL^T u = -b$. Si spezza in due passi triangolari:

$$\underbrace{Ly = -b}_{1. \text{ Forward substitution}} \longrightarrow \underbrace{L^T u = y}_{2. \text{ Backward substitution}}$$

Ogni sistema triangolare si risolve in $\mathcal{O}(n^2)$ con la sostituzione in avanti/indietro, contro $\mathcal{O}(n^3)$ di Gauss completo.

Schema visivo della fattorizzazione:



3.2 Il fenomeno del fill-in

La matrice originale $-A$ associata alla griglia fisica è estremamente *sparsa*: avendo al più 5 elementi non nulli per riga (derivanti dallo stencil a 5 punti), essa contiene solo $\mathcal{O}(N^2)$ entrate non nulle. Tuttavia, durante il processo di fattorizzazione di Cholesky, il fattore triangolare inferiore L tende a “riempirsi” di nuovi elementi che originariamente erano nulli.

Definizione 3.1 (Fill-in). Un elemento in posizione (i, j) del fattore di Cholesky L (con $i > j$) è chiamato **fill-in** se:

$$L_{ij} \neq 0 \quad \text{ma} \quad (-A)_{ij} = 0$$

Si tratta cioè di un'entrata che era originariamente nulla in $-A$ ma che diventa diversa da zero a causa dei calcoli di eliminazione.

Perché si genera il fill-in?

Durante l'eliminazione parziale della colonna k , gli elementi della sottomatrice attiva rimasta vengono aggiornati tramite la formula di eliminazione gaussiana:

$$A_{ij}^{(k+1)} = A_{ij}^{(k)} - \frac{A_{ik}^{(k)} A_{kj}^{(k)}}{A_{kk}^{(k)}} \quad (i, j > k)$$

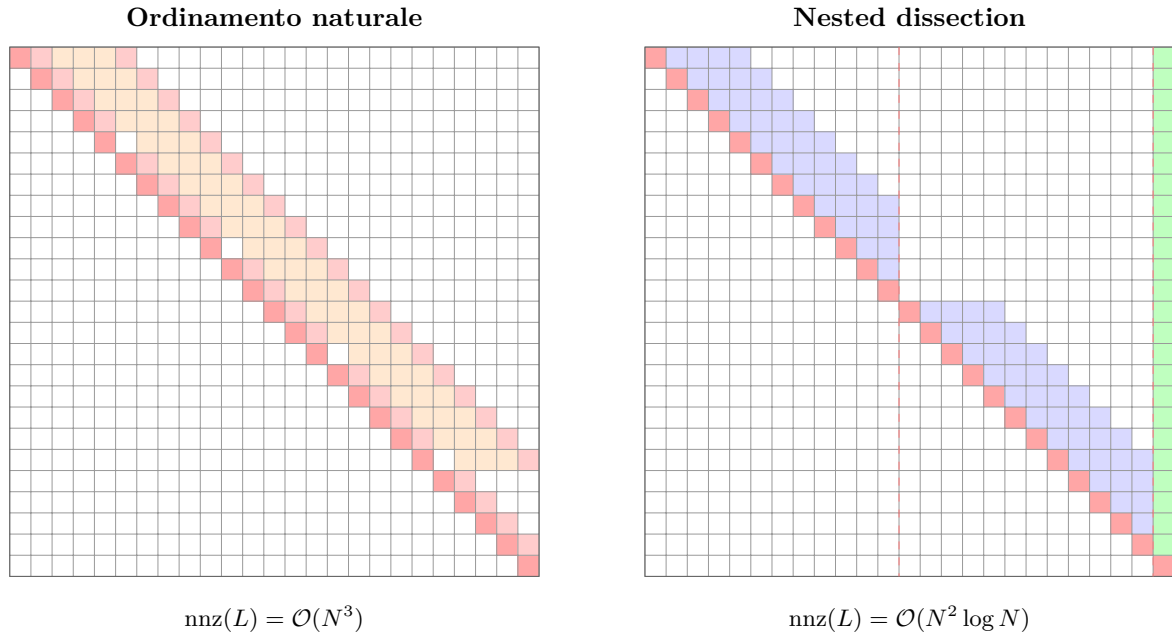
Se l'elemento di partenza $A_{ij}^{(k)}$ è nullo, ma sia $A_{ik}^{(k)}$ che $A_{kj}^{(k)}$ sono diversi da zero (ovvero il nodo k connetteva originariamente i nodi i e j), l'aggiornamento produce un valore non nullo $A_{ij}^{(k+1)} \neq 0$. Questo è un nuovo elemento di fill-in. In termini di teoria dei grafi, eliminando un nodo k , creiamo una connessione (una cricca) tra tutti i suoi vicini non ancora eliminati.

Impatto dell'ordinamento delle incognite:

Il numero di elementi di fill-in non è intrinseco alla matrice, ma dipende drasticamente dall'ordine con cui decidiamo di eliminare le equazioni (nodi della griglia).

	Ordinamento Naturale	Nested Dissection
Larghezza di banda β	N	N (non strutturata)
Entrate non nulle di $-A$	$\approx 5N^2$	$\approx 5N^2$
Elementi non nulli in L : $\text{nnz}(L)$	$\mathcal{O}(N^3)$ (molto denso)	$\mathcal{O}(N^2 \log N)$ (molto sparso)
RAM stimata per $N = 512$	≈ 1 GB	≈ 10 MB
Tempo di fattorizzazione	$\mathcal{O}(N^4)$	$\mathcal{O}(N^3)$

Illustrazione del fill-in (per $N = 5$, matrice associata 25×25):



Sinistra: Fill-in con ordinamento naturale (arancione) — riempie interamente la banda tra le diagonali esterne.

Destra: Fill-in con nested dissection (blu) — i blocchi V_1 e V_2 sono isolati; solo il separatore comune V_S (verde) genera fill-in, limitato però a poche righe/colonne finali.

3.3 La Nested Dissection: Teoria, Algoritmo e Vantaggi

La **Nested Dissection** (Dissezione Annidata) è una tecnica euristica di tipo *divide-et-impera* introdotta da Alan George nel 1973 per determinare un ordinamento ottimale delle incognite che minimizzi drasticamente la generazione di fill-in durante la fattorizzazione di matrici sparse simmetriche definite positive.

3.3.1 Principio di Funzionamento (La scomposizione del Grafo)

L'algoritmo interpreta la struttura di sparsità della matrice $-A$ come un grafo non orientato $G = (V, E)$, in cui i vertici V rappresentano le incognite (i nodi della griglia di calcolo) e gli archi E indicano le relazioni di adiacenza fisica date dallo stencil.

La scomposizione avviene attraverso i seguenti passaggi ricorsivi:

1. **Trovare un Separatore di Vertici (V_S):** Si identifica un sottoinsieme di vertici relativamente piccolo $V_S \subset V$ la cui rimozione scollega il grafo G in due sotto-grafi disgiunti, V_A e V_B , aventi dimensioni il più possibile bilanciate. Per definizione, non esiste alcun arco in E che connetta direttamente un nodo appartenente a V_A con un nodo appartenente a V_B .
2. **Imporre l'Ordinamento:** Si decide di numerare (e quindi eliminare) le variabili in modo sequenziale: prima tutte le incognite del blocco V_A , successivamente tutte quelle del blocco V_B , e soltanto alla fine le incognite che compongono il separatore comune V_S .
3. **Ricorsione geometrica:** Il medesimo procedimento viene applicato in maniera ricorsiva sui sotto-grafi indotti da V_A e V_B , individuando sotto-separatori interni e suddividendo ulteriormente i domini fino a raggiungere una dimensione minima prefissata (caso base).

3.3.2 Perché la Nested Dissection minimizza il fill-in?

La chiave dell'efficacia dell'algoritmo risiede nell'isolamento strutturale. Poiché non esistono connessioni dirette tra V_A e V_B , la rimozione (eliminazione) delle variabili appartenenti a V_A non può indurre alcun aggiornamento numerico (e quindi alcun fill-in) sulle variabili appartenenti a V_B , e viceversa. Il fill-in è rigorosamente confinato all'interno di ciascun sotto-blocco locale, e si propaga verso l'esterno esclusivamente in direzione dei nodi del separatore V_S . Poiché i nodi del separatore vengono eliminati per ultimi, essi non possono trasmettere o propagare il fill-in a ritroso nei blocchi già completati.

Per grafi planari (come la nostra griglia di calcolo 2D), il *Teorema del Separatore Planare di Lipton-Tarjan* garantisce che ogni grafo di n nodi può essere ripartito in modo bilanciato da un separatore di dimensione limitata da $\mathcal{O}(\sqrt{n})$. Questa proprietà limita superiormente la crescita del fill-in, garantendo che:

- Il numero di elementi non nulli nel fattore L si riduca da $\mathcal{O}(N^3)$ (ordinamento naturale) a soli $\mathcal{O}(N^2 \log N)$.
- Il numero di operazioni in virgola mobile (*flops*) necessarie per calcolare Cholesky scenda da $\mathcal{O}(N^4)$ a $\mathcal{O}(N^{2.5})$ o $\mathcal{O}(N^3)$ estremamente sparsi.

Visualizzazione dell'ordinamento su una griglia 7×7 (49 nodi):

La figura sottostante mostra la numerazione finale prodotta dalla Nested Dissection. Il separatore verticale principale (colonna 4, in rosso) divide la griglia in due metà indipendenti e viene eliminato per ultimo (indici da 43 a 49). Le due metà vengono a loro volta tagliate orizzontalmente dai sotto-separatori secondari (riga 4, in azzurro e giallo). I sotto-blocchi elementari (in grigio e verde) sono numerati per primi, risultando mutuamente isolati.



3.3.3 Due scuole di pensiero: Approccio su Grafo vs Approccio Geometrico

Quando si deve implementare la Nested Dissection, ci si trova di fronte a un bivio algoritmico su come individuare materialmente i separatori. Esistono due approcci diametralmente opposti:

1. L'approccio puramente su grafo (Topologico):

È l'approccio più generale e astratto. L'algoritmo (come nel famoso pacchetto software METIS) riceve in input *esclusivamente* la lista delle connessioni (gli archi) tra i nodi, senza sapere assolutamente nulla della loro posizione fisica nello spazio. Per trovare il separatore, l'algoritmo deve "esplorare" il grafo alla cieca usando complesse euristiche di ricerca (come la bisezione spettrale, che calcola gli autovettori della matrice Laplaciana del grafo, oppure iterative visite in ampiezza BFS).

Pro: È universale e funziona su qualsiasi geometria, persino su griglie deformate, triangolari o non strutturate.

Contro: È computazionalmente pesantissimo. Per una griglia $N \times N$, il solo leggere tutti gli archi costa $\mathcal{O}(N^2)$ operazioni di memoria, e le euristiche di taglio aggiungono ulteriore lentezza e complessità, restituendo spesso partizioni imperfette.

2. L'approccio geometrico (Spaziale - Quello usato nel nostro progetto!):

Sfrutta un'intuizione fisica potentissima: se conosciamo a priori le coordinate spaziali (x, y) dei nodi, possiamo ignorare del tutto la lista degli archi! Sapendo che la griglia è regolare, sappiamo per certo che due nodi fisicamente distanti non possono essere direttamente collegati dallo stencil. Il separatore può quindi essere individuato tracciando un semplicissimo "piano mediano" nello spazio geometrico.

Invece di navigare un grafo immenso saltando da un nodo all'altro, l'algoritmo calcola il valore medio della coordinata spaziale lungo un asse (ad esempio, calcola la mediana delle x) e taglia la griglia di netto: i nodi che si trovano a sinistra della mediana formano il blocco V_1 , quelli a destra formano V_2 , e quelli che cadono esattamente sulla linea mediana formano il nostro separatore V_S .

Perché l'approccio geometrico vince a mani basse sulla nostra griglia

Nel nostro progetto per l'equazione del calore, la griglia di calcolo è un quadrato perfettamente cartesiano e uniforme. Utilizzare un complesso approccio su grafo in questo contesto sarebbe come usare un navigatore satellitare per andare dalla propria camera da letto alla propria cucina! L'approccio geometrico ci garantisce tre vantaggi incolmabili:

- **Ignorare la connettività:** Il nostro Modulo 2 non carica mai il file `connectivity.txt`, risparmiando un'enorme quantità di tempo di I/O e di allocazione RAM.
- **Velocità pura:** Trovare il taglio richiede unicamente la lettura sequenziale delle coordinate per calcolarne la media, portando a tempi di esecuzione ultra-efficienti ($\mathcal{O}(N^2 \log N)$ calcolato su semplici interi), completando il riordinamento di un milione di nodi in una frazione di secondo.
- **Robustezza e perfezione:** Il taglio geometrico è matematicamente perfetto. Divide sempre la griglia in due metà speculari senza doversi affidare a euristiche probabilistiche o approssimazioni.

3.3.4 Come si applica nel Progetto (C++ e Python)

Nel progetto concreto, l'algoritmo viene implementato proprio seguendo l'innovativo approccio geometrico, sfruttando la conoscenza delle coordinate spaziali dei nodi (x, y) lette agilmente dal file `coords.txt`.

- **In `task2_reorder.cpp` (Calcolo degli indici):** L'algoritmo calcola la scomposizione geometrica in modo iterativo per evitare l'overhead di ricorsione profonda. A ogni passo,

si considera il bounding box dei nodi attivi e si calcola la mediana geometrica lungo la coordinata spaziale più estesa:

$$\text{mid_val} = \frac{\min(x) + \max(x)}{2}$$

I nodi situati in prossimità di questo valore di soglia vengono marcati come parte del separatore principale (impostando il flag `separatore = true` all'interno della struttura dati `StackItem`). Questo processo viene gestito inserendo le restanti metà (sinistra e destra) in una struttura a pila (*stack*) per la successiva scomposizione ricorsiva.

- **In `task3_assemble.cpp` (Costruzione del sistema):** Una volta terminata la scomposizione, si generano il vettore di permutazione globale `perm` e il suo inverso `inv_perm`. Le equazioni vengono quindi riordinate prima dell'assemblaggio, mappando la matrice originale A direttamente nella sua versione permutata simmetricamente:

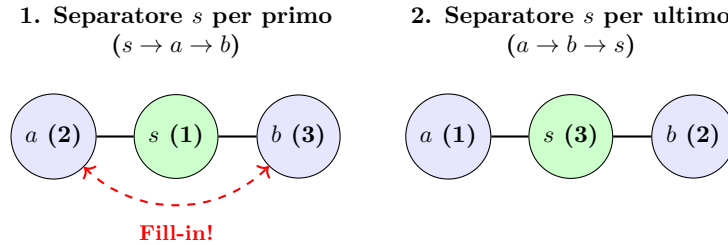
$$A_{\text{perm}} = P A P^T$$

e aggiornando opportunamente il termine noto $b_{\text{perm}} = P b$.

- **In `solver.ipynb` (Fattorizzazione e Risoluzione):** La risoluzione del sistema lineare si riduce ad applicare la scomposizione di Cholesky sulla matrice riordinata $-A_{\text{perm}} = LL^T$. Poiché la matrice è estremamente sparsa e confinata grazie alla Nested Dissection, la memoria allocata e i tempi di esecuzione per la scomposizione e la successiva *forward/backward substitution* risultano quasi istantanei anche per griglie molto fitte (fino a $N = 1024$).

3.4 Esempio grafico con un sistema minimo a 3 nodi

Per comprendere matematicamente come la disposizione influenzi il calcolo, consideriamo una riga minima di 3 nodi $a - s - b$, dove il nodo centrale s fa da separatore tra i nodi esterni a e b (non direttamente connessi tra loro). Vediamo cosa succede con due diverse numerazioni:



- **Ordinamento 1 (Separatore per primo - $s \rightarrow a \rightarrow b$):** Eliminando il nodo s (indice 1), i suoi vicini a (2) e b (3) si trovano accoppiati matematicamente nell'aggiornamento. Si genera una connessione fittizia (fill-in) tra a e b .

$$-A = \begin{pmatrix} \bullet & \bullet & \bullet \\ \bullet & \bullet & 0 \\ \bullet & 0 & \bullet \end{pmatrix} \implies L = \begin{pmatrix} \bullet & 0 & 0 \\ \bullet & \bullet & 0 \\ \bullet & \bullet & \bullet \end{pmatrix} \quad (\text{Fill-in in posizione } (3, 2))$$

- **Ordinamento 2 (Separatore per ultimo - $a \rightarrow b \rightarrow s$):** Eliminiamo prima a (1) e b (2). Poiché non sono connessi tra loro, l'eliminazione di a aggiorna solo s , e l'eliminazione di b aggiorna solo s . Non viene creata alcuna connessione fittizia tra a e b .

$$-A = \begin{pmatrix} \bullet & 0 & \bullet \\ 0 & \bullet & \bullet \\ \bullet & \bullet & \bullet \end{pmatrix} \implies L = \begin{pmatrix} \bullet & 0 & 0 \\ 0 & \bullet & 0 \\ \bullet & \bullet & \bullet \end{pmatrix} \quad (\text{Nessun fill-in, struttura intatta})$$

4 Modulo 1 — task1_grid.cpp: griglia e grafo

4.1 Obiettivo e panoramica del flusso

Il Modulo 1 è il punto di ingresso dell'intera pipeline. Il suo compito è trasformare un singolo parametro intero N in due file di testo che descrivono completamente la struttura discreta del dominio:

- `coords.txt`: per ogni nodo interno della griglia, il suo identificatore lineare $\phi(i,j)$, gli indici (i,j) , e le coordinate fisiche (x,y) nel dominio $[0,1]^2$.
- `connectivity.txt`: la lista di adiacenza del grafo, una riga per arco non orientato.

Tutti gli altri moduli dipendono da questi file; se cambia N , task1 è l'unico da rieseguire per rigenerare tutto.

4.2 Lettura dell'argomento da riga di comando: `argc`, `argv` e `stoi`

Listing 1: Parsing robusto dell'argomento N

```

1 int main(int argc, char* argv[]) {
2     int N = 0;
3     if (argc >= 2) {
4         try {
5             N = stoi(argv[1]); // converte la stringa "32" nell'intero
6                               32
7         } catch (...) {
8             N = 0; // se l'utente passa "abc", N rimane 0
9         }
10    }
11    // Fallback interattivo: ciclo che insiste finche' N>0 o utente
12    // preme 'q'
13    while (N <= 0) {
14        cout << "Inserire N (oppure digita 'q' per uscire): ";
15        string input_str;
16        if (!(cin >> input_str) || input_str == "q" || input_str == "Q")
17            {
18                return 0; // Uscita pulita
19            }
20        try {
21            N = stoi(input_str);
22        } catch (...) {
23            N = 0; // Se input non e' un numero, N torna 0 e il ciclo
24                // ripete
25        }
26    }
27 }
```

Perché `stoi` e non `atoi`? Entrambe convertono una stringa in intero, ma con comportamenti molto diversi in caso di input errato:

- `atoi("abc")` ritorna silenziosamente 0 senza alcun segnale di errore, rendendo difficile il debugging.
- `std::stoi("abc")` lancia l'eccezione `std::invalid_argument`, che il blocco `try/catch` intercetta pulitamente, impostando $N = 0$ e forzando l'ingresso nel ciclo interattivo `while`.

Perché il fallback iterativo? Rende il programma perfettamente autonomo sia da riga di comando (per la pipeline automatizzata del notebook) sia per un utente umano che lo avvia

senza argomenti o sbagliando l'input, permettendo di correggere l'errore senza dover riavviare l'eseguibile (o uscendo comodamente premendo q).

Cosa sono `argc` e `argv`? Sono i parametri standard di `main` in C++:

- `argc` (Argument Count): numero di stringhe passate sulla riga di comando, incluso il nome del programma. Eseguendo `./task1 32`, `argc = 2`.
- `argv` (Argument Vector): array di puntatori a stringa C. `argv[0]` è il nome del programma, `argv[1]` è il primo argomento (la stringa "32").

4.3 Creazione della directory di output: `mkdir` e il codice 0755

Listing 2: Creazione sicura della directory output/

```
1 #include <sys/stat.h>
2 static const string OUTPUT_DIR = "output/";
3 // ...
4 mkdir(OUTPUT_DIR.c_str(), 0755);
```

Perché `static const string` invece di `#define`? La macro `#define` è una semplice sostituzione testuale eseguita dal preprocessore, senza tipo e senza scope. Una `static const string` è invece un oggetto C++ con tipo, visibile solo nell'unità di traduzione corrente (`static`), non modificabile (`const`) e utilizzabile con le funzioni della Standard Library.

Cosa fa `.c_str()`? La funzione della POSIX `mkdir()` è un'interfaccia del sistema operativo (sistema C) e accetta solo puntatori a stringa C (`const char*`), non oggetti `std::string` del C++ moderno. Il metodo `.c_str()` restituisce un puntatore a carattere che punta all'array interno della stringa.

Cosa significa 0755? È un numero ottale (prefisso 0) che codifica i permessi UNIX:

- 7 (`rw`x) per il proprietario: lettura, scrittura, esecuzione.
- 5 (`r-x`) per il gruppo: lettura e accesso, ma non scrittura.
- 5 (`r-x`) per tutti gli altri: idem.

Se la directory `output/` esiste già, `mkdir` fallisce silenziosamente (non è un errore fatale): il programma continua e sovrascrive semplicemente i file esistenti.

4.4 La funzione `getNodeId`: `inline` e formula row-major

Listing 3: Mappatura 2D → 1D, funzione inline

```
1 // i, j vanno da 1 a N. Restituisce un ID in [0, N^2-1].
2 inline int getNodeId(int i, int j, int N) {
3     return (i - 1) * N + (j - 1);
4 }
```

Perché `inline`? Normalmente, quando si chiama una funzione, il processore deve:

1. Salvare i registri correnti (*stack frame*).
2. Saltare all'indirizzo della funzione.
3. Eseguire il corpo.
4. Ripristinare i registri e tornare al chiamante.

Questo *overhead* è costoso se la funzione è microscopica e viene chiamata milioni di volte (una per ogni nodo nei doppi cicli `for` di `task1` e `task3`). La direttiva `inline` suggerisce al

compilatore di sostituire ogni chiamata con il corpo della funzione, eliminando completamente l'overhead. In pratica, il compilatore con ottimizzazioni abilitato (-O2) lo farebbe comunque (*inlining automatico*), ma la direttiva rende l'intenzione esplicita.

Perché la formula $(i-1) \cdot N + (j-1)$? Si tratta del classico *ordine row-major*: i nodi sono numerati riga dopo riga da sinistra a destra.

- La riga i (con i che va da 1 a N) inizia all'offset $(i-1) \cdot N$. Sottraiamo 1 perché i nostri indici partono da 1, ma gli ID devono partire da 0.
- Dentro la riga, la colonna j aggiunge un offset $j-1$.

Esempio 4.1. Per $N = 3$: il nodo in posizione riga 2, colonna 3 ha $\phi(2, 3) = (2-1) \cdot 3 + (3-1) = 3 + 2 = 5$, ovvero è il sesto elemento del vettore (indicizzazione da 0).

4.5 Generazione coordinate e precisione floating point

Listing 4: Scrittura delle coordinate con precisione controllata

```

1 coords_file << fixed << setprecision(8);
2
3 for (int i = 1; i <= N; ++i) {
4     for (int j = 1; j <= N; ++j) {
5         int n = getNodeId(i, j, N);
6         double x = (double)i / (N + 1);
7         double y = (double)j / (N + 1);
8         coords_file << n << " " << i << " " << j
9                     << " " << x << " " << y << "\n"; // '\n' NON '<<endl
10    }
11 }
```

Perché il cast `(double)i`? Senza cast, la divisione $i / (N+1)$ sarebbe una divisione intera (entrambi `int`), che per $i < N + 1$ darebbe sempre 0. Il cast forza la divisione in virgola mobile.

Perché `setprecision(8)` con `fixed`? Per default, `cout` usa la notazione scientifica e arrotonda aggressivamente. Impostando `fixed` e 8 cifre decimali, i valori come $1/(N+1)$ vengono scritti nella forma 0.03030303 anziché 0.030303, garantendo che il Modulo 2 li rilegga senza perdita di informazione.

Perché `'\n'` e non `std::endl`? `std::endl` scrive il carattere di newline e poi forza il *flush* del buffer di scrittura del sistema operativo, causando una scrittura su disco dopo ogni singola riga. Per $N = 512$ sono 262.144 righe: usare `endl` moltiplica il tempo di I/O per decine di volte rispetto a `'\n'`, che lascia al sistema operativo la libertà di accumulare dati nel buffer interno e scriverli su disco in blocchi efficienti.

4.6 Costruzione del grafo con *reserve* e approccio “in avanti”

Listing 5: Lista di adiacenza con pre-allocazione

```

1 vector<vector<int>> adj_list(num_nodes);
2 for (int k = 0; k < num_nodes; ++k)
3     adj_list[k].reserve(4); // PRE-ALLOCA 4 slot per ogni nodo
4
5 for (int i = 1; i <= N; ++i) {
6     for (int j = 1; j <= N; ++j) {
7         int u = getNodeId(i, j, N);
8         if (j < N) { // collega a destra (j+1)
9             int v = getNodeId(i, j+1, N);
10            adj_list[u].push_back(v);

```

```

11         adj_list[v].push_back(u); // bidirezionale
12     }
13     if (i < N) { // collega in alto (i+1)
14         int v = getNodeId(i+1, j, N);
15         adj_list[u].push_back(v);
16         adj_list[v].push_back(u);
17     }
18 }
19 }

```

Perché `.reserve(4)`? Un `std::vector` inizia con capacità zero e raddoppia ogni volta che si esaurisce. Questo significa riallocazioni e copie costose. Chiamare `reserve(4)` prima di qualsiasi `push_back` dice al vettore: “alloca subito 4 slot”. Dato che ogni nodo interno ha al massimo 4 vicini, questo elimina completamente le riallocazioni.

Perché l’approccio “in avanti” (solo su e a destra)? Il grafo è non orientato: ogni arco $\{u, v\}$ deve comparire una volta sola nel file finale. Collegando il nodo corrente solo con il vicino in alto ($j+1$) e a destra ($i+1$), ogni coppia viene considerata esattamente una volta durante il ciclo. Aggiornare però entrambe le liste (`adj_list[u]` e `adj_list[v]`) mantiene la bidirezionalità nella struttura dati in memoria.

4.7 Scrittura degli archi senza duplicati: la condizione $u < v$

Listing 6: Filtro $u < v$ per evitare archi duplicati nel file

```

1 for (int u = 0; u < num_nodes; ++u)
2     for (int v : adj_list[u])
3         if (u < v) // ogni coppia non ordinata compare una sola volta
4             conn_file << u << " " << v << "\n";

```

La lista di adiacenza è bidirezionale: se `adj_list[u]` contiene `v`, allora `adj_list[v]` contiene `u`. Se iterassimo su tutti gli elementi senza filtro, ogni arco verrebbe scritto due volte. La condizione $u < v$ garantisce che per ogni coppia $\{u, v\}$ venga scritta solo la versione con l’ID più piccolo prima: l’altra (`v, u` con $v > u$) viene saltata.

4.8 Gestione degli Errori (Fail) e Output Generati

Casi di fallimento (Fail):

- Se si inserisce un input testuale non valido da terminale, il Modulo 1 intercetta l’eccezione ed entra in un ciclo di *fallback iterativo* fino all’inserimento di un valore $N > 0$ o del carattere `q` per uscire.
- Se la cartella `output/` non può essere creata (es. permessi negati), il programma stampa a terminale l’errore `impossibile creare output/coords.txt` ed esce con codice 1.

Output generati (output/):

- **coords.txt**: Formato `id i j x y` (es. `0 1 1 0.25 0.25`). Rappresenta le coordinate fisiche di ciascun nodo, indispensabili per valutare la funzione sorgente e le condizioni al bordo.
- **connectivity.txt**: Formato `edge_id u v` (es. `0 0 1`). Specifica gli archi del grafo, usati dal Modulo 2 per analizzare la topologia di connessione.

5 Modulo 2 — task2_reorder.cpp: nested dissection

5.1 Panoramica: approccio geometrico vs approccio su grafo

Esistono due filosofie per implementare la nested dissection:

- **Approccio su grafo:** si esplora la lista di adiacenza del grafo cercando un separatore bilanciato tramite algoritmi come BFS/DFS o partitioning di Kernighan–Lin. È generale ma costoso e complesso.
- **Approccio geometrico (adottato nel progetto):** per un dominio 2D regolare come la nostra griglia $N \times N$, il separatore ottimale del grafo coincide esattamente con il separatore geometrico: la riga (o colonna) mediana del dominio. Si opera direttamente sulle coordinate spaziali (i, j) , senza mai leggere il file `connectivity.txt`.

Il vantaggio è notevole: complessità $\mathcal{O}(N^2 \log N)$, accesso sequenziale ai dati (*cache-friendly*), implementazione deterministica e priva di parametri euristici.

5.2 La struttura Node: perché non usare una coppia di interi?

Listing 7: Struttura Node con campi nominati

```

1 struct Node {
2     int id;           // ID naturale  $\phi(i, j) = (i-1)*N + (j-1)$ 
3     int i;           // indice di riga [1..N]
4     int j;           // indice di colonna [1..N]
5     double x;        // coordinata fisica  $x = i/(N+1)$ 
6     double y;        // coordinata fisica  $y = j/(N+1)$ 
7 };

```

Si sarebbe potuto usare `std::pair<int,int>` o anche `std::tuple`, ma la `struct` con campi nominati è preferibile perché il codice diventa leggibile: `node.i` e `node.j` comunicano l'intenzione, mentre `p.first` e `p.second` non lo fanno. Il compilatore produce lo stesso codice macchina in entrambi i casi; la differenza è puramente di leggibilità e manutenibilità.

Perché tenere sia (i, j) che (x, y) ? L'algoritmo di taglio usa gli indici interi (i, j) (più robusti numericamente: nessun confronto in floating point), mentre le coordinate fisiche (x, y) sono necessarie se si volesse estendere l'algoritmo a domini non uniformi o se il task3 ne avesse bisogno.

5.3 La struttura StackItem e il flag is_separator

Listing 8: StackItem: l'astrazione chiave dell'algoritmo iterativo

```

1 struct StackItem {
2     vector<Node> S;    // il sottoinsieme di nodi da elaborare
3     int axis;         // asse di taglio: 0 = X (usa i), 1 = Y (usa j)
4     bool is_separator; // true = aggiungere all'ordinamento senza
                        // partizionare
5 };

```

Il campo `is_separator` è la chiave dell'algoritmo iterativo. Nella versione ricorsiva, il separatore V_S viene aggiunto all'ordinamento *dopo* che la ricorsione su V_1 e V_2 è completata. Nella versione iterativa non esiste questo concetto di “dopo la ricorsione”. La soluzione è inserire V_S nello stack con il flag `is_separator=true`: quando verrà estratto, il ciclo `while` lo riconoscerà e lo copierà direttamente nell'ordinamento finale, senza partizionarlo ulteriormente.

5.4 Versione iterativa con `std::stack`: perché non la ricorsione?

L'algoritmo di nested dissection è intrinsecamente ricorsivo. La versione naïve in C++ sarebbe:

Listing 9: Versione RICORSIVA — pericolosa per N grandi

```

1 void nd_ricorsiva(vector<Node> S, int axis, vector<int>& ord) {
2     // ... partiziona in V1, VS, V2 ...
3     nd_ricorsiva(V1, 1-axis, ord); // frame con vettore di N^2/2 nodi
4     nd_ricorsiva(V2, 1-axis, ord); // idem
5     for (auto& n : VS) ord.push_back(n.id);
6 }

```

Il problema: ogni chiamata ricorsiva alloca un frame sullo *stack di sistema*, che è tipicamente limitato a 1–8 MB. Per una griglia $N = 1024$, la profondità della ricorsione è $\log_2(1024^2) = 20$ livelli. Ogni frame contiene un `vector<Node>` con fino a $1024^2/2 \approx 500.000$ nodi (circa 20 MB). La memoria richiesta è enorme e lo stack di sistema va in overflow (*Stack Overflow*, non il sito web).

La soluzione: `std::stack` allocato sull'heap. La libreria standard di C++ `<stack>` implementa una struttura LIFO (Last In, First Out) i cui elementi vengono allocati sull'*heap*, la memoria dinamica di processo che in Linux è praticamente illimitata (limitata dalla RAM e dallo swap). L'algoritmo diventa così:

Listing 10: Versione ITERATIVA con stack esplicito — sicura

```

1 void nested_dissection_geometrica(const vector<Node>& nodi_iniziali,
2                                   int axis_iniziale,
3                                   vector<int>& ordering)
4 {
5     stack<StackItem> stk; // stack sull'heap
6     stk.push({nodi_iniziali, axis_iniziale, false});
7
8     while (!stk.empty()) {
9         StackItem item = move(stk.top()); // (1) move: trasferimento
10        // senza copia
11        stk.pop();
12
13        if (item.is_separator) { // (2) separatore: copia e
14            // basta
15            for (const auto& n : item.S)
16                ordering.push_back(n.id);
17            continue;
18        }
19        if (item.S.empty()) continue; // (3) caso base vuoto
20        if (item.S.size() == 1) { // (4) caso base singolo
21            ordering.push_back(item.S[0].id);
22            continue;
23        }
24        // ... partizionamento e push ...
25    }
26 }

```

(1) Perché `move(stk.top())`? L'operazione `stk.top()` restituisce un riferimento all'elemento in cima allo stack. Senza `move`, scrivere `StackItem item = stk.top()` eseguirebbe una *copia profonda* del vettore `S`, che per $N = 512$ può contenere decine di migliaia di elementi. `std::move` converte il valore in un *rvalue reference*, trasferendo la proprietà della memoria interna del vettore senza copiare i dati: l'operazione è $\mathcal{O}(1)$ invece di $\mathcal{O}(\text{size})$.

(2–4) Perché tre casi base? Rispecchiano i casi della ricorsione:

- `is_separator`: il separatore è già stato calcolato in un'iterazione precedente.

- `S.empty()`: possibile se il partizionamento produce un sottoinsieme vuoto (es. griglia con N pari e tutti i nodi su un lato della mediana).
- `S.size()==1`: un singolo nodo non ha da partizionare.

5.5 Il partizionamento geometrico: mediana intera e reserve pessimistico

Listing 11: Calcolo della mediana e suddivisione in V1, VS, V2

```

1 // Range dell'asse di taglio
2 int min_val = (item.axis == 0) ? item.S[0].i : item.S[0].j;
3 int max_val = min_val;
4 for (const auto& node : item.S) {
5     int val = (item.axis == 0) ? node.i : node.j;
6     if (val < min_val) min_val = val;
7     if (val > max_val) max_val = val;
8 }
9 int mid_val = (min_val + max_val) / 2; // divisione intera: mediana
   geometrica
10
11 vector<Node> V1, V2, VS;
12 V1.reserve(item.S.size()); // (!) pessimistico, ma evita riallocazioni
13 V2.reserve(item.S.size());
14 VS.reserve(item.S.size());
15
16 for (const auto& node : item.S) {
17     int val = (item.axis == 0) ? node.i : node.j;
18     if (val < mid_val) V1.push_back(node);
19     else if (val > mid_val) V2.push_back(node);
20     else VS.push_back(node); // separatore geometrico
21 }

```

Perché la mediana intera $(\text{min_val} + \text{max_val}) / 2$? La divisione intera di due `int` in C++ esegue il troncamento verso zero (floor). Per indici sempre positivi, questa è esattamente la mediana del range. La scelta degli *indici interi* (i, j) al posto delle coordinate floating point (x, y) garantisce confronti esatti, senza errori di arrotondamento che potrebbero classificare erroneamente nodi al confine.

Perché `.reserve(item.S.size())` su tutti e tre i vettori? È una stima *pessimistica*: nel caso peggiore, tutti i nodi potrebbero finire in un singolo vettore. Riservare la dimensione massima possibile evita qualsiasi riallocazione durante i `push_back`, al costo di sprecare temporaneamente della memoria. Dato che i tre vettori sono locali alla singola iterazione e vengono deallocati appena estratti dallo stack, il consumo di memoria è sempre sotto controllo.

5.6 L'alternanza degli assi e la logica LIFO dello stack

Listing 12: Inserimento in ordine inverso: LIFO garantisce FIFO nell'elaborazione

```

1 int next_axis = 1 - item.axis; // alterna: 0->1, 1->0
2 // Vogliamo elaborare: V1 prima, V2 dopo, VS per ultimo.
3 // Lo stack e' LIFO: l'ultimo inserito e' il primo estratto.
4 // Quindi inseriamo in ordine INVERSO: VS, V2, V1.
5 stk.push({move(VS), item.axis, true}); // VS inserito per primo =
   estratto per ultimo
6 stk.push({move(V2), next_axis, false}); // V2 inserito secondo =
   estratto per secondo
7 stk.push({move(V1), next_axis, false}); // V1 inserito per ultimo =
   estratto per primo

```

Perché 1 - item.axis? È un trucco elegante per alternare un flag binario: se `axis = 0` (asse X), l'espressione vale 1 (asse Y); se `axis = 1`, vale 0. L'alternanza garantisce che i tagli si alternino tra orizzontale e verticale ad ogni livello della gerarchia, producendo blocchi sempre bilanciati in entrambe le dimensioni.

La logica LIFO in dettaglio: Lo stack è Last In, First Out. Affinché l'elaborazione avvenga nell'ordine $V1 \rightarrow V2 \rightarrow VS$, bisogna inserire nell'ordine inverso $VS \rightarrow V2 \rightarrow V1$. $V1$ in cima allo stack viene estratto per primo; VS in fondo viene estratto solo quando sia $V1$ che $V2$ (e tutte le loro sotto-partizioni ricorsive) sono stati completamente elaborati. Questo è precisamente il comportamento che avremmo con la ricorsione.

Nota: si usa `move(VS)` anche qui per evitare la copia del vettore durante il `push` nello stack.

5.7 Gestione degli Errori (Fail) e Output Generato

Casi di fallimento (Fail):

- **File mancanti:** Se `connectivity.txt` o `coords.txt` non esistono, il programma segnala Errore: impossibile aprire... ricordando di eseguire `task1_grid` ed esce con codice 1.
- **Input inattesi:** Se le righe dei file lette non coincidono con il parametro N atteso (es. `coords.txt` generato con un N diverso), vengono stampati dei warning o gestiti i bound array.

Output generato (output/): Il Modulo 2 produce il file `ordering.txt`.

Listing 13: Salvataggio: nuova posizione -> vecchio ID

```
1 for (size_t k = 0; k < ordering.size(); ++k)
2   out_file << k << " " << ordering[k] << "\n";
```

Il file prodotto ha una riga per ogni nodo, nel formato: `nuova_pos vecchio_id`.

Esempio per $N = 2$ (4 nodi):

```
0 2    (la nuova posizione 0 corrisponde al vecchio nodo 2)
1 0
2 3
3 1    (il vecchio nodo 1 = separatore, messo per ultimo)
```

Il Modulo 3 legge questo file e costruisce **due array di permutazione complementari**:

- `perm[k] = old_id`: dato il nuovo indice k (riga/colonna nella matrice riordinata), dammi il vecchio ID fisico del nodo.
- `inv_perm[old_id] = k`: dato il vecchio ID fisico (usato per trovare i vicini), dammi il suo indice nella matrice riordinata (usato per scrivere nell'entrata corretta).

6 Modulo 3 — task3_assemble.cpp: sistema lineare

6.1 Panoramica: cosa fa il Modulo 3 e perché è il più complesso

Il Modulo 3 è il cuore computazionale della pipeline C++. Riceve in input:

- La geometria della griglia (`coords.txt`): coordinate fisiche di ogni nodo.
- Opzionalmente, la permutazione calcolata (`ordering.txt`).

E produce in output:

- **A.txt**: le entrate non nulle della matrice sparsa in formato COO (triplette riga, colonna, valore).
- **b.txt**: il vettore del termine noto **b**.

La complessità sta nel tradurre lo stencil differenziale in entrate di matrice, applicare la permutazione di riordinamento “al volo”, e gestire le condizioni al bordo di Dirichlet.

6.2 Costanti fisiche: `static const` e pre-calcolo dei coefficienti

Listing 14: Definizione delle costanti fisiche

```

1 static const double KAPPA = 0.01;
2
3 inline double f_sorgente(double x, double y) {
4     return exp(-10.0 * (x * x + y * y));
5 }
6
7 double h          = 1.0 / (N + 1);
8 double coeff_diag = -4.0 * KAPPA / (h * h); // -4k/h^2: coeff.
           diagonale
9 double coeff_fuori_diag = KAPPA / (h * h);      // +k/h^2: coeff.
           off-diag

```

Perché `static const double` e non `#define KAPPA 0.01`?

- `#define` è una macro del preprocessore: sostituisce testualmente `KAPPA` con il numero 0.01 ovunque nel file. Non ha tipo, non ha scope, non può essere ispezionata dal debugger.
- `static const double` è una vera variabile C++ con tipo (`double`), scope (il file corrente grazie a `static`), e che il compilatore può ottimizzare in modo sicuro.

Perché pre-calcolare `coeff_diag` e `coeff_fuori_diag`? L'espressione κ/h^2 viene usata migliaia di volte nel ciclo di assemblaggio. Ricalcolarla ogni iterazione sarebbe (in linea di principio) ridondante. Il compilatore potrebbe ottimizzarla automaticamente, ma pre-calcolarla esplicitamente rende il codice più leggibile ed è una best practice.

6.3 Le due mappe di permutazione: il contratto tra Modulo 2 e Modulo 3

Listing 15: Costruzione di `perm` e `inv_perm`

```

1 vector<int> perm(num_nodi);      // perm[k]          = vecchio ID del nodo
           in posizione k
2 vector<int> inv_perm(num_nodi); // inv_perm[old]     = nuova posizione del
           vecchio nodo old
3
4 if (usa_reorder) {
5     int new_id, old_id;
6     while (file_ordering >> new_id >> old_id) {
7         perm[new_id] = old_id;

```



```

8     inv_perm[old_id] = new_id;
9     }
10 } else {
11     // Ordinamento naturale: permutazione identita'
12     for (int k = 0; k < num_nodi; ++k) { perm[k] = k; inv_perm[k] = k; }
13 }

```

Perché due vettori distinti e non uno solo? Le due mappe svolgono ruoli opposti nel ciclo di assemblaggio:

- **perm:** si itera su $k = 0, 1, \dots, N^2 - 1$ (posizioni nella matrice riordinata). Per assemblare la riga k , bisogna sapere *quale nodo fisico* occupa quella posizione: `old_id = perm[k]` fornisce il suo ID naturale, con cui si accede alle coordinate (x, y) per calcolare $f(x, y)$.
- **inv_perm:** una volta noti i vicini fisici del nodo k (identificati dai loro ID naturali), bisogna sapere *in quale colonna* della matrice riordinata compaiono: `k_vicino = inv_perm[old_id_vicino]` fornisce l'indice di colonna corretto per l'entrata off-diagonal.

Senza `inv_perm`, per ogni vicino si dovrebbe fare una ricerca lineare su `perm`, portando la complessità dell'assemblaggio da $\mathcal{O}(N^2)$ a $\mathcal{O}(N^4)$.

6.4 Gli array di spostamento di[] e dj[]: il ciclo compatto sui 4 vicini

Listing 16: Spostamenti dello stencil codificati in array

```

1 const int di[] = {-1, +1, 0, 0}; // spostamenti lungo l'asse i (riga)
2 const int dj[] = { 0, 0, -1, +1}; // spostamenti lungo l'asse j (
   colonna)
3 // d=0: vicino a sinistra (i-1, j)
4 // d=1: vicino a destra (i+1, j)
5 // d=2: vicino in basso (i, j-1)
6 // d=3: vicino in alto (i, j+1)
7
8 for (int d = 0; d < 4; ++d) {
9     int ni = ri + di[d]; // riga del vicino
10    int nj = rj + dj[d]; // colonna del vicino
11    // ...
12 }

```

Perché array di spostamenti invece di 4 blocchi if separati? Il codice avrebbe potuto trattare i 4 vicini con 4 blocchi condizionali indipendenti. La versione con array è superiore perché:

- **Eliminazione della duplicazione:** la logica di controllo bordo e di scrittura dell'entrata è scritta una sola volta e vale per tutti e 4 i vicini.
- **Estendibilità:** per aggiungere uno stencil a 9 punti basterebbe allungare gli array.
- **Ottimizzabilità:** il compilatore può vettorizzare il ciclo `for (d=0;d<4;d++)` più facilmente di 4 blocchi separati.

6.5 Il ciclo di assemblaggio completo con gestione del bordo

Listing 17: Ciclo principale: dalla riga k della matrice riordinata alle entrate

```

1 vector<Triplet> triplets;
2 triplets.reserve(5 * num_nodi); // al piu' 5 entrate per riga (1 diag +
   4 off-diag)
3 vector<double> b(num_nodi, 0.0);
4

```



```

5  for (int k = 0; k < num_nodi; ++k) {
6      int old_id = perm[k];           // nodo fisico in posizione k
7      int ri = coords[old_id].i;     // indice di riga fisico
8      int rj = coords[old_id].j;     // indice di colonna fisico
9      double x = coords[old_id].x;
10     double y = coords[old_id].y;

11
12     triplets.push_back({k, k, coeff_diag}); // A[k,k] = -4κ/h^2
13     b[k] = -f_sorgente(x, y);             // b[k] = -f(x,y)
14
15     for (int d = 0; d < 4; ++d) {
16         int ni = ri + di[d];
17         int nj = rj + dj[d];
18
19         if (ni < 1 || ni > N || nj < 1 || nj > N) {
20             // Vicino di bordo: valore noto u0, passa al termine noto
21             b[k] -= coeff_fuori_diag * get_u0(ni*h, nj*h, scelta);
22         } else {
23             // Vicino interno: entrata off-diagonal A[k, k_vicino] = +κ/h^2
24             int k_vicino = inv_perm[ getNodeId(ni, nj, N) ];
25             triplets.push_back({k, k_vicino, coeff_fuori_diag});
26         }
27     }
28 }

```

Perché $b[k] -= \text{coeff_fuori_diag} * u_0$? È l'applicazione matematica delle condizioni di Dirichlet. Scritta l'equazione per il nodo k con un vicino di bordo a temperatura u_0 :

$$\frac{\kappa}{h^2}(\dots + u_0 + \dots) - \frac{4\kappa}{h^2}u_k = -f_k$$

Spostando il termine noto $\frac{\kappa}{h^2}u_0$ al secondo membro:

$$\frac{\kappa}{h^2}(\dots) - \frac{4\kappa}{h^2}u_k = -f_k - \frac{\kappa}{h^2}u_0$$

Quindi si sottrae ($-$) il termine di bordo dal termine noto.

6.6 La struttura Triplet e il formato COO di A.txt

Listing 18: Struttura Triplet e scrittura A.txt con alta precisione

```

1  struct Triplet { int row; int col; double val; };
2
3  triplets.reserve(5 * num_nodi); // 1 diag + al piu' 4 off-diag per riga
4
5  ofstream file_A(OUTPUT_DIR + "A.txt");
6  file_A << fixed << setprecision(10); // 10 cifre: preserva i double
7  for (const auto& t : triplets)
8      file_A << t.row << " " << t.col << " " << t.val << "\n";

```

Perché `setprecision(10)` per A.txt ma solo 8 per coords.txt? Le coordinate geometriche sono rapporti interi $i/(N+1)$ con valori come 0.03030303; 8 cifre sono più che sufficienti. I coefficienti della matrice invece sono espressioni come $\kappa/h^2 = 0.01 \cdot (N+1)^2$ che per $N = 512$ valgono circa 2.7×10^6 : servono più cifre significative per preservare la precisione numerica nella risoluzione del sistema.

Perché `reserve(5 * num_nodi)`? È una stima del numero totale di entrate non nulle: ogni nodo contribuisce 1 entrata diagonale e al massimo 4 off-diagonali. La pre-allocazione evita le riallocazioni del vettore durante i `push_back`, riducendo il tempo di assemblaggio.

Nota sul file A.txt: ogni entrata off-diagonal compare *due volte* nel file: la riga per (k, k') e la riga per (k', k) , perché la matrice è simmetrica e il ciclo analizza ogni nodo indipendentemente. Il Modulo 4 (Python) gestisce automaticamente questo tramite `coo_matrix`, che somma le entrate con gli stessi indici.

6.7 Le condizioni al bordo: switch e funzioni di verifica

Listing 19: La funzione `get_u0` parametrizzata con switch

```

1 double get_u0(double x, double y, int scelta) {
2     switch (scelta) {
3         case 0: return 0.0;           // omogenee
4         case 1: return 10.0;         // costante
5         case 2: return x;             // gradiente
6         case 3: return sin(M_PI * x); // sinusoidale
7         case 4: return x * x - y * y; // armonica
8         case 5: return (y >= 1.0 - 1e-9) ? 1.0 : 0.0; // shock termico
9         default: return 0.0;
10    }
11 }
```

Perché switch invece di una catena if-else? Per interi, il compilatore implementa `switch` tramite una *jump table*: un array di puntatori a codice, indicizzato direttamente dal valore di `scelta`. L'esecuzione è $\mathcal{O}(1)$ indipendentemente dal numero di casi. Una catena `if-else` invece viene valutata sequenzialmente: nel caso peggiore (case 5) richiederebbe 5 confronti.

Perché $1e-9$ nel caso 5? Il confronto `y >= 1.0` su un `double` è pericoloso: il valore $y = j/(N+1)$ per $j = N + 1$ dovrebbe essere esattamente 1, ma rappresentato in virgola mobile potrebbe essere 0.999999... o 1.000000001 a causa degli errori di arrotondamento. La tolleranza $1e-9$ rende il confronto robusto.

Scopo fisico e Sanity Checks sperimentali: Le diverse condizioni al bordo implementate non sono solo varianti fisiche, ma costituiscono la base per i **Sanity Checks sperimentali**, eseguiti in Python per validare in automatico la correttezza della discretizzazione (se $f = 0$):

1. **Omogenea** ($u_0 = 0$): *Caso Zero*. La soluzione u deve essere ovunque esattamente 0.
2. **Costante** ($u_0 = 10$): il bordo è a 10° . La soluzione deve essere uniformemente 10° ovunque nella piastra.
3. **Gradiente lineare** ($u_0 = x$): temperatura crescente linearmente.
4. **Sinusoidale** ($u_0 = \sin(\pi x)$): test visivo (heatmap ondulata).
5. **Armonica** ($u_0 = x^2 - y^2$): poiché $\nabla^2(x^2 - y^2) = 0$, la soluzione numerica calcolata dal programma **deve coincidere perfettamente** col valore teorico di u_0 in ogni nodo interno, a meno degli errori di arrotondamento (residuo e discostamento inferiori a 10^{-10}).
6. **Shock termico** ($u_0 = 1_{y=1}$): stress-test di gradienti forti.

6.8 Gestione degli Errori (Fail) e Output Generati

Casi di fallimento (Fail):

- **Argomenti mancanti o flag errati:** Se N non è fornito o il flag (se presente) non è `-reorder`, il programma stampa a schermo l'uso corretto (Usage) ed esce con codice 1.
- **File di input assenti:** Se mancano `coords.txt` (o `ordering.txt` in caso di `-reorder`), viene generato un errore bloccante che ricorda l'esecuzione di `task1` o `task2`.

- **Disallineamento del parametro N :** Se il file `coords.txt` contiene un numero di righe diverso da N^2 , il parser rileva l'incongruenza e interrompe l'esecuzione (**lette X righe, ma N richiede Y nodi**).

Output generati (output/):

- **A.txt:** Formato **riga colonna valore** (es. 0 0 -400.0 e 0 1 100.0). Rappresenta le entrate non nulle della matrice di rigidezza. Le entrate fuori diagonale sono salvate due volte (matrice simmetrica), ma il modulo Python le aggregherà correttamente in `coo_matrix`.
- **b.txt:** Formato **valore** per riga (es. -0.135). Rappresenta il vettore dei termini noti b . È già comprensivo dei contributi derivati dalle condizioni di Dirichlet al bordo.

7 Modulo 4 — solver.ipynb: risoluzione Python

7.1 Dal formato COO a CSC: perché due formati diversi?

Listing 20: Lettura di A.txt in COO e conversione in CSC

```

1 import numpy as np
2 import scipy.sparse as sp
3 import scipy.sparse.linalg as spla
4 from sksparse.cholmod import cholesky as cholmod_cholesky
5 from scipy.sparse import csc_matrix
6
7 def read_A(path):
8     data = np.loadtxt(path)           # legge tutte le triplette in un
      array
9     rows = data[:, 0].astype(int)
10    cols = data[:, 1].astype(int)
11    vals = data[:, 2]
12    n     = max(rows.max(), cols.max()) + 1
13    A     = sp.coo_matrix((vals, (rows, cols)), shape=(n, n))
14    return A.tocsc() # converte in Compressed Sparse Column

```

Perché il formato COO? COO (Coordinate Format) è il formato più semplice per costruire una matrice sparsa: si passa semplicemente una lista di triplette (**riga**, **colonna**, **valore**). Il vantaggio cruciale nel nostro caso: quando la stessa coppia (**riga**, **colonna**) compare più volte, `coo_matrix` somma i valori duplicati durante la conversione. Questo gestisce automaticamente il fatto che il C++ scrive ogni entrata off-diagonal due volte (sia (k, k') che (k', k)): dopo la somma, ogni entrata ha il valore corretto.

Perché convertire in CSC? CHOLMOD (la libreria di fattorizzazione di Cholesky) richiede il formato CSC (Compressed Sparse Column). CSC è ottimale per operazioni colonna-per-colonna, come la fattorizzazione di Cholesky stessa. La struttura interna di `csc_matrix` per una matrice $n \times n$ con `nnz` elementi non nulli è:

- `data[nnz]`: i valori non nulli, colonna per colonna.
- `indices[nnz]`: l'indice di riga di ogni valore.
- `indptr[n+1]`: `indptr[j]` dice dove inizia la colonna j nell'array `data`. `indptr[j+1] - indptr[j]` è il numero di elementi non nulli nella colonna j .

Questo permette di accedere a tutti gli elementi di una colonna in tempo $\mathcal{O}(\text{nnz di quella colonna})$ invece di scansionare tutta la matrice.

7.2 Lettura del vettore b.txt e delle coordinate

Listing 21: Lettura del termine noto e delle coordinate

```

1 def read_b(path):
2     return np.loadtxt(path) # restituisce un array NumPy 1D
3
4 def read_coords(path):
5     data = np.loadtxt(path) # colonne: id, i, j, x, y
6     n     = int(data[:, 0].max()) + 1
7     xs    = np.zeros(n)
8     ys    = np.zeros(n)
9     for row in data:
10        k      = int(row[0]) # ID del nodo
11        xs[k]  = row[3]      # coordinata x
12        ys[k]  = row[4]      # coordinata y
13    return xs, ys

```

Perché `np.loadtxt`? È la funzione NumPy ottimizzata per leggere file testuali di numeri: internamente usa buffer grandi e conversioni di tipo efficienti, molto più veloci di leggere riga per riga con Python puro.

7.3 La fattorizzazione di Cholesky con CHOLMOD: API e scelte

Listing 22: La funzione `my_cholesky` con l'API aggiornata

```

1 def my_cholesky(neg_A):
2     # neg_A = -A e' SPD (Simmetrica Definita Positiva)
3     # La nuova API di sksparse restituisce una tupla (L, perm)
4     L, _ = cholmod_cholesky(neg_A, order='natural', lower=True)
5     return csc_matrix(L)

```

Perché si passa `-A` e non `A`? La matrice A del nostro sistema è *definita negativa* (tutti gli autovalori sono < 0). Cholesky funziona solo su matrici *definite positive*. Passando $-A$ si inverte il segno di tutti gli autovalori: la matrice diventa SPD e Cholesky è applicabile.

Perché `order='natural'`? CHOLMOD internamente può ri-riordinare le colonne della matrice per minimizzare il fill-in (usando AMD, METIS, ecc.). Specificando `'natural'` gli diciamo: *usa l'ordine già presente nella matrice*. Questo è fondamentale perché il Modulo 2 ha già calcolato l'ordinamento ottimale tramite nested dissection; lasciare che CHOLMOD ri-riordini a sua volta annullerebbe il nostro lavoro.

L'API ha cambiato valore di ritorno: nelle versioni recenti di `sksparse`, `cholesky()` restituisce una tupla `(L, perm)`: il fattore L e il vettore di permutazione usato internamente. Con `order='natural'` il secondo elemento è la permutazione identità, quindi lo scartiamo con `_`.

7.4 La risoluzione del sistema: `spsolve_triangular` vs `spsolve`

Listing 23: Forward e backward substitution esplicite

```

1 def solve(A, b):
2     neg_A = (-A).tocsc()           # -A e' SPD
3     t0 = time.perf_counter()       # timer ad alta risoluzione
4     L = my_cholesky(neg_A)         # -A = L L^T
5     t_chol = time.perf_counter() - t0
6
7     # Passo 1 (forward): L y = -b
8     y = spla.spsolve_triangular(L, -b, lower=True)
9     # Passo 2 (backward): L^T u = y
10    u = spla.spsolve_triangular(L.T.tocsr(), y, lower=False)
11    return u, L, t_chol

```

Perché `spsolve_triangular` e non `spsolve`? `scipy.sparse.linalg.spsolve` è un risolutore generale che, non sapendo che la matrice è triangolare, userebbe metodi più costosi (ad es. LU factorization interna). `spsolve_triangular` è specializzato per matrici triangolari e usa la sostituzione in avanti/indietro: algoritmo $\mathcal{O}(\text{nnz}(L))$ invece di $\mathcal{O}(N^3)$.

Perché `L.T.tocsr()`? L è una `csc_matrix` (ottimale per accesso per colonna). La sua trasposta L^T è triangolare superiore. `spsolve_triangular` con `lower=False` si aspetta una `csc_matrix` (ottimale per accesso per riga, formato naturale per la sostituzione all'indietro che procede riga per riga). La conversione `.tocsr()` è $\mathcal{O}(\text{nnz})$.

Perché `time.perf_counter()`? Python offre diverse funzioni per misurare il tempo:

- `time.time()`: restituisce il tempo di parete (wall-clock) in secondi, ma con risoluzione bassa (tipicamente 10 ms su Linux).

- `time.perf_counter()`: usa il contatore hardware ad alta risoluzione del processore (nanosecondo), ed è il metodo raccomandato per il benchmarking.

7.5 Automazione della pipeline: subprocess e iniezione via stdin

Listing 24: `run_pipeline`: orchestrazione dei moduli C++ da Python tramite `argv`

```

1 import subprocess
2 from pathlib import Path
3 import numpy as np
4
5 ROOT = Path("../").resolve() # root del progetto
6 OUTPUT = ROOT / "output"    # cartella degli output
7
8 def run_pipeline(N, use_reorder, bordo=0):
9     # task1: genera la griglia
10    subprocess.run([str(ROOT / 'task1'), str(N)],
11                  cwd=str(ROOT), capture_output=True, text=True, check=
12                    True)
13
14    # task2: calcola la permutazione
15    if use_reorder:
16        subprocess.run([str(ROOT / 'task2')],
17                        cwd=str(ROOT), capture_output=True, text=True,
18                          check=True)
19
20    # task3: assembla la matrice
21    cmd = [str(ROOT / "task3"), str(N)]
22    if use_reorder:
23        cmd.append("--reorder")
24    cmd.append(str(bordo))
25
26    subprocess.run(cmd, cwd=str(ROOT), capture_output=True, text=True,
27                  check=True)
28
29    # [SANITY CHECK] Integrità della permutazione
30    if use_reorder:
31        ordering = np.loadtxt(ROOT / "output/ordering.txt", dtype=int)
32        assert len(np.unique(ordering)) == len(ordering) == N*N, "
33            Permutazione invalida!"

```

Perché passare parametri via `argv` e non da `stdin` interattivo? Nelle iterazioni iniziali del progetto, il Modulo 3 chiedeva la scelta del bordo richiedendo un input da tastiera. Questo approccio è fallimentare in un contesto di automazione *Data Science* con Jupyter Notebook, costringendo l'iniezione forzata dell'input (es. `input=b"0"`). Passando `str(bordo)` direttamente come parametro terminale (tramite l'argomento in coda in `argv`), il programma viene eseguito in modalità batch "un-shot" in modo estremamente robusto ed elegante.

Perché `subprocess.run` invece di `os.system`? `os.system` lancia un processo tramite la shell del sistema operativo (bash/sh): lento, insicuro, e non restituisce informazioni strutturate. `subprocess.run` lancia il processo direttamente e permette di catturare `stdout/stderr` e verificare il codice di uscita.

Il Sanity Check finale su Python: Si nota l'asserzione (`assert`) alla fine della pipeline. Legge il file prodotto dal `task2` per validarne l'integrità matematica al volo. Se, per un bug nel C++, un ID di nodo venisse scartato o duplicato nell'ordinamento, l'operatore `np.unique` lo paleserebbe bloccando immediatamente l'esecuzione ed evitando di inquinare la fattorizzazione con un sistema fallato.

7.6 Interpretazione dei grafici e analisi empirica

Spy plot: `plt.spy(L)` disegna un pixel per ogni elemento non nullo di L . Per l'ordinamento naturale: struttura densa e regolare a “blocchi pieni” sotto la diagonale — fill-in enorme. Per la nested dissection: struttura molto più sparsa, con interi quadranti vuoti, e il separatore (gli ultimi nodi elaborati) che compare come una colonna/riga densa solo nell'angolo in basso a destra.

Grafico log-log: tracciando $\log(\text{nnz}(L))$ su asse y e $\log(N)$ su asse x , la pendenza della retta approssimativa indica l'esponente della legge di scala:

- Ordinamento naturale: pendenza ≈ 3 ($\text{nnz}(L) = \mathcal{O}(N^3)$).
- Nested dissection: pendenza ≈ 2 con leggera curvatura ($\mathcal{O}(N^2 \log N)$).

I dati sperimentali ottenuti ($N=32 \rightarrow 512$) confermano esattamente questa analisi asintotica.

Il limite fisico della RAM: il crash OOM per $N = 1024$

Per l'**ordinamento naturale** con $N = 1024$ (circa 1 milione di incognite), il fill-in teorico è $\mathcal{O}(N^3) = \mathcal{O}(10^9)$ elementi. Ogni `double` occupa 8 byte, quindi il fattore L richiederebbe circa $8 \times 10^9 / 10^9 = 8$ GB di RAM contigua. Su una macchina con 6 GB, il kernel Linux attiva l'*OOM Killer* e termina il processo.

Con la **Nested Dissection** ($\mathcal{O}(N^2 \log N) \approx 10^7$ elementi), il fattore L occupa circa 80 MB: banalmente gestibile. L'intero sistema di 10^6 equazioni viene risolto in meno di 10 secondi.

8 Riepilogo: dalla teoria al codice

Concetto matematico	Dove appare nel codice	File
Mappa $\phi(i, j) = (i - 1)N + (j - 1)$	<code>getNodeId(i, j, N)</code>	<code>task1_grid.cpp</code>
Stencil 5 punti ∇^2	Loop su <code>di[]</code> , <code>dj[]</code>	<code>task3_assemble.cpp</code>
Condizioni di Dirichlet	Ramo <code>if</code> (fuori dominio)	<code>task3_assemble.cpp</code>
Mediana geometrica	<code>mid_val = (min+max)/2</code>	<code>task2_reorder.cpp</code>
Separatore V_S	<code>struct StackItem,</code> <code>separatore=true</code>	<code>task2_reorder.cpp</code>
Permutazione <code>perm/inv_perm</code>	<code>vector<int> perm,</code> <code>inv_perm</code>	<code>task3_assemble.cpp</code>
Fattorizzazione $-A = LL^T$	<code>cholesky(-A,</code> <code>order="natural")</code>	<code>solver.ipynb</code>
Forward/backward substitution	<code>spsolve_triangular(L,</code> <code>...)</code>	<code>solver.ipynb</code>
Fill-in $\mathcal{O}(N^3)$ vs $\mathcal{O}(N^2 \log N)$	Grafico log-log di <code>nnz(L)</code>	<code>solver.ipynb</code>

Riepilogo del flusso end-to-end

1. **task1** genera la griglia $N \times N$ e il grafo di adiacenza.
2. **task2** calcola la permutazione nested dissection (ordinamento ottimale).
3. **task3** costruisce la matrice A e il vettore b nel nuovo ordinamento.
4. **solver.ipynb** fattorizza $-A = LL^T$ e risolve $LL^T u = -b$.
5. Il fill-in di L è $\mathcal{O}(N^2 \log N)$ (nested dissection) contro $\mathcal{O}(N^3)$ (ordinamento naturale): per $N = 512$ è un risparmio di circa $100\times$.

9 Appendice: Domande Ricorrenti d'Esame

In questa sezione vengono analizzate e fornite le risposte alle domande più frequenti poste dai docenti durante la discussione del progetto.

9.1 1. Struttura generale del codice: come sono suddivisi i programmi e cosa fanno i diversi moduli?

Il progetto adotta un'architettura modulare suddivisa in quattro fasi logiche distinte (tre implementate in C++ e una orchestrata in Python), che comunicano *esclusivamente* tramite file di testo intermedi. Questa scelta garantisce disaccoppiamento e testabilità.

- **Modulo 1 (`task1_grid.cpp`):** Genera la griglia computazionale e il grafo logico. Crea il file delle coordinate (`coords.txt`) assegnando a ogni nodo interno un ID progressivo $\phi(i, j)$, e il file delle connessioni topologiche (`connectivity.txt`).
- **Modulo 2 (`task2_reorder.cpp`):** Riceve il file delle coordinate ed esegue l'algoritmo di *Nested Dissection Geometrica* per calcolare un riordinamento ottimale dei nodi, esportando la mappa risultante in `ordering.txt`.
- **Modulo 3 (`task3_assemble.cpp`):** Riceve i nodi e l'ordinamento. Calcola i coefficienti fisici dello stencil differenziale, mappa le incognite nelle nuove posizioni logiche e assembla le equazioni (matrice sparsa `A.txt` in formato COO e termine noto `b.txt`), applicando coerentemente le condizioni al bordo scelte dall'utente.
- **Modulo 4 (`solver.ipynb`):** Acquisisce i file testuali, assembla la matrice finale in memoria in formato CSC (compresso per colonne), lancia la fattorizzazione di Cholesky tramite librerie specializzate (`CHOLMOD`) e visualizza i risultati grafici dell'analisi.

9.2 2. Ordinamento: approccio a grafi vs geometrico e dettagli dell'algoritmo

Approccio scelto: Nel progetto è stato consapevolmente adottato l'approccio **geometrico**. Per un dominio 2D regolare, usare complessi algoritmi di visita su grafo (esplorando liste di adiacenza) è inefficiente e superfluo. Grazie all'approccio geometrico, il `task2` non ha nemmeno bisogno di leggere il grafo (`connectivity.txt`), ma si basa esclusivamente sulle coordinate spaziali, tagliando di netto il dominio a metà calcolando banali mediane intere lungo gli assi.

Come è scritto l'algoritmo: Al posto della naturale ricorsione (che per griglie grandi come 1024×1024 causerebbe immediatamente un *Stack Overflow*), l'algoritmo è stato implementato in modo **iterativo** avvalendosi di uno `std::stack` esplicito allocato sull'heap.

1. Estrae un set di nodi dalla pila e stabilisce l'asse su cui effettuare il taglio (riga o colonna, alternati ad ogni livello).
2. Calcola la coordinata mediana per la dimensione scelta.
3. Partiziona l'array di nodi in V_1 (coordinata minore), V_2 (maggiore) e V_S (uguale alla mediana).
4. Poiché lo stack funziona con logica LIFO (Last In, First Out) e noi desideriamo che il separatore V_S sia valutato per ultimo (in modo che finisca in fondo alla matrice), l'inserimento nello stack avviene a ritroso: per primo viene impilato V_S , poi V_2 , infine V_1 (che si troverà in cima e verrà esaminato subito).

Complessità:

- *Temporale:* $\mathcal{O}(N^2 \log N)$ per il partizionamento sequenziale. Più efficiente delle omologhe euristiche a grafo (spesso limitate a complessità cubiche $\mathcal{O}(N^3)$ se non ottimizzate).
- *Spaziale:* $\mathcal{O}(N^2)$ dominata dall'immagazzinamento delle coordinate e dello stack esplicito.

- *Possibili varianti:* Se il dominio fosse un rettangolo non uniforme (anziché $N \times N$), converrebbe modificare il partizionamento scegliendo sempre l'asse più allungato (invece di alternare rigidamente X e Y) per garantire tagli sempre ottimali (*longest-edge bisection*).

9.3 3. Il generatore di matrici sparse: assemblaggio e struttura printata

Come funziona (Modulo 3): Si itera sequenzialmente sulla matrice riordinata (da $k = 0$ a $k = N^2 - 1$). Usando un vettore inverso (`perm`), si risale al nodo fisico spaziale originale e si individuano i suoi 4 vicini (Alto, Basso, Dx, Sx). Se il vicino è nel dominio, si aggiunge l'entrata matrice fuori-diagonale; altrimenti si modifica il termine noto se c'è condizione di bordo non omogenea. Il file prodotto, `A.txt`, è in un banale formato a tre colonne: **riga colonna valore**. La particolarità sta nel fatto che il programma C++ processa e stampa tutti i vicini di ogni nodo; ciò significa che l'arco tra i e j viene stampato due volte (simmetria).

Passaggio in Python e Spy Plot: In Python, il comando `scipy.sparse.coo_matrix` ingurgita il file grezzo e *somma automaticamente le entrate duplicate* (eliminando il problema della simmetria ridondante), prima di comprimere tutto nel formato CSC. La struttura che viene stampata a schermo (la funzione `spy(L)`) mostra, accendendo un pixel per ogni elemento non nullo, l'entità del famigerato "fill-in". Nell'ordinamento *Naturale*, lo spy plot mostra masse monolitiche dense attorno alla diagonale. Con la *Nested Dissection*, lo spy plot rivela una bellissima struttura geometrica rada: interi quadranti della matrice sono vuoti, mentre si intravedono strette bande non nulle concentrate principalmente lungo i bordi esterni dei vari blocchi separatori.

9.4 4. Soluzione grafica (Color Plot/Heatmap)

Essendo le equazioni riordinate per minimizzare lo spreco computazionale, il vettore risultato u del sistema lineare è a sua volta "mescolato". Prima di essere elaborato, u viene ritrasformato secondo la permutazione naturale inversa, così da far coincidere ogni indice lineare alla propria posizione 2D originale (x, y) . Tramite `matplotlib` viene generato un *color plot* (heatmap). Date le equazioni del sistema, la rappresentazione grafica mostra esplicitamente un nucleo caldo asimmetrico nel vertice inferiore-sinistro (dove è situata la sorgente termica gaussiana) e una sfumatura fredda (blu intenso) procedendo verso i margini della piastra per assecondare la condizione di Dirichlet in cui i bordi si comportano come pozzi di calore a 0° .

9.5 5. Mostrare il codice in azione

Per dimostrare il flusso all'esame, l'approccio ideale è utilizzare una griglia piccolissima (es. $N = 4$):

1. `./task1 4` (genera 16 nodi).
2. Mostrare rapidamente `coords.txt`, sottolineando l'identificatore progressivo da 0 a 15.
3. `./task2`. Aprendo `ordering.txt`, evidenziare come i primi nodi stampati non siano più $0, 1, 2 \dots$ ma i nodi d'angolo periferici (che formano il gruppo V_1 distaccato), provando materialmente l'avvenuta dissezione.
4. `./task3 4 -reorder`. Mostrare il menu interattivo e l'esportazione pulita in `A.txt`.

9.6 6. Generazione via Intelligenza Artificiale (Meta-Commento Documentale)

In alcune occasioni, il docente ha esplicitamente richiesto di sottoporre un estratto del documento di design a un modello generativo, per testarne la chiarezza: "Se dessi questo documento in pasto a un'IA, produrrebbe codice C++ corretto?". L'idea alla base è che il documento non deve limitarsi a descrivere "a grandi linee", bensì fornire le **specifiche funzionali assolute**:

input/output file, invarianti strutturali (come il "non caricare `connectivity.txt` nel Modulo 2"), le strutture dati (es. lo "Stack Esplicito") e i casi di fallimento o fallback (come la lettura da tastiera del parametro mancante). Grazie all'articolazione quasi pre-codice delle sezioni logiche qui fornite, e all'utilizzo degli pseudocodici nel modulo di *design*, la transcodifica risulta inequivocabile e meccanica.

9.7 7. Dettagli Implementativi C++ e Scelte Sintattiche

Durante la discussione o l'analisi del codice, possono emergere domande specifiche sulle scelte puramente sintattiche adottate in C++. Ecco le principali:

- **La direttiva `inline`** (es. `inline int getId(...)`): Questa direttiva suggerisce al compilatore di espandere il corpo della funzione direttamente nel punto in cui viene chiamata, evitando l'*overhead* (costo computazionale) tipico dell'invocazione di funzione (salvataggio dei registri, salto nello stack, ripristino). Trattandosi di una semplice formula aritmetica $\phi(i, j) = (i - 1)N + (j - 1)$ richiamata milioni di volte all'interno dei cicli `for` annidati in `task1` e `task3`, renderla `inline` abbassa significativamente i tempi di esecuzione e permette al compilatore di applicare ottimizzazioni più aggressive (*loop unrolling* e vettorizzazione).
- **Gestione degli input da terminale (`argc`, `argv` e `std::stoi`)**: Il passaggio di parametri essenziali come N avviene tramite argomenti da riga di comando. L'array `argv` contiene stringhe in formato C (`char*`). Per convertire la stringa che rappresenta la dimensione della griglia in un vero numero intero su cui fare operazioni aritmetiche, è fondamentale usare la funzione della Standard Library `std::stoi` (String TO Integer). Al contrario del vecchio `atoi` ereditato dal C, `std::stoi` offre sicurezza sui tipi lanciando un'eccezione (`std::invalid_argument`) se l'utente fornisce lettere o formati non validi invece di un numero.
- **Il metodo `.reserve()` sui `std::vector`**: Nei vari task, spesso allochiamo array dinamici che conterranno migliaia di elementi (es. le coordinate dei nodi o lo stack nel task 2). Utilizzare preventivamente `nodi.reserve(N*N)` istruisce il vettore ad allocare in blocco tutta la memoria necessaria fin dall'inizio. Questo evita le continue riallocazioni interne (e conseguenti copie di tutti gli elementi già inseriti) che il vettore sarebbe altrimenti costretto a fare ogni volta che si esaurisce la sua capacità di default durante ripetuti `push_back()`, garantendo un inserimento a tempo costante rigoroso $\mathcal{O}(1)$.
- **Scrittura su file: `\n` vs `std::endl`**: Durante l'esportazione di matrici sparse (`A.txt`), il codice deve scrivere milioni di righe su disco. L'uso sistematico di `'\n'` anziché `std::endl` è una scelta mirata alle *performance*: `std::endl` non si limita ad andare a capo, ma forza ogni volta il *flush* del buffer del sistema operativo sul disco fisico. Questo causerebbe colli di bottiglia catastrofici (I/O bound). Scrivendo `\n`, lasciamo che il sistema operativo decida il momento migliore per riversare i blocchi di dati su disco, riducendo i tempi di I/O di ordini di grandezza.